

A PROJECT REPORT ON

**FARMER ADVISORY REPORT GENERATOR USING
SOIL, WEATHER & CROP DATA USING GENAI**

SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE
DEGREE

OF

BACHELOR OF ENGINEERING

IN

INFORMATION TECHNOLOGY

SUBMITTED BY

MR. OMKAR SHELKE

EXAM NO: B400430410

MRS. TRUPTI KANTHALE

EXAM NO: B400430420

MR. VAIBHAV VAVHAL

EXAM NO: B400430421

MR. AMAR VARPE

EXAM NO: B400430422

UNDER THE GUIDANCE OF

DR. SUNIL L. BANGARE



Sinhgad Institutes

DEPARTMENT OF INFORMATION TECHNOLOGY

STES'S SINHGAD ACADEMY OF ENGINEERING

KONDHWA BK, PUNE 411048

ACADEMIC YEAR 2025-26



Sinhgad Institutes

CERTIFICATE

This is to certify that the project report entitles

**FARMER ADVISORY REPORT GENERATOR USING SOIL, WEATHER & CROP DATA
USING GENAI**

Submitted by

MR. OMKAR SHELKE

EXAM NO: B400430410

MRS. TRUPTI KANTHALE

EXAM NO: B400430420

MR. VAIBHAV VAVHAL

EXAM NO: B400430421

MR. AMAR VARPE

EXAM NO: B400430422

is a bonafide work has been carried out by them under the supervision of **Dr. Sunil L. Bangare** and it is approved for the partial fulfilment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (INFORMATION TECHNOLOGY).

Dr. Sunil L. Bangare

Guide

Department of
Information Technology

Dr. S.S. Kulkarni

H.O.D.

Department of
Information Technology

Dr. M. S. Rohokale

Principal

SAE, Pune

Place: **Pune**

Date: / / 2026

External Examiner

Dr. Sunil L. Bangare

Project Coordinator

ACKNOWLEDGEMENT

I hereby take this opportunity to record my sincere thanks and heartily gratitude to **Dr. Sunil L. Bangare** for his useful guidance and making available to me his intimate knowledge and experience in making “**Smart Farmar Advisory Report Generator Using Soil, Weather & Crop Data Using GenAI**” as a preparation of report in respect thereof. I am also thankful to my respective **HOD Dr. S.S. Kulkarni** of my Information Technology department. I express my special thanks and heartily gratitude to my respective staff members for inspiring me throughout the completion of this system. The acknowledge will be incomplete if I don't record sense of gratitude to my principal. I also express my sincere thanks to all those i.e., the management, lab assistants, my friends and family who have provided me valuable guidance towards the completion of this presentation as a part of the syllabus of the course.

I express my sincere gratitude towards co-operative department who have provided me with valuable assistance and requirements for the presentation.

Project Group Members:

MR. OMKAR SHELKE

EXAM NO: B400430410

MRS. TRUPTI KANTHALE

EXAM NO: B400430420

MR. VAIBHAV VAVHAL

EXAM NO: B400430421

MR. AMAR VARPE

EXAM NO: B400430422

ABSTRACT

Abstract — Agriculture is increasingly affected by climate variability, unpredictable weather conditions, and crop diseases that reduce productivity and threaten food security. Farmers often rely on fragmented tools for monitoring weather, soil conditions, and disease detection, which leads to inefficient decision making. To address these challenges, this research presents a web-based Farmer Advisory Report Generator that integrates soil data, weather information, and crop disease detection using Artificial Intelligence (AI) and Generative Artificial Intelligence (GenAI).

The proposed system collects real-time environmental data from IoT-based sensors such as NodeMCU, soil moisture sensors, and DHT11 temperature–humidity sensors. In addition, the system retrieves real-time weather and forecast data from the Open-Meteo API. Farmers can also upload crop leaf images, which are analyzed using a Convolutional Neural Network (CNN) to detect plant diseases. The detected results are processed through a Generative AI Large Language Model (LLM) such as Gemini to generate comprehensive advisory reports including disease diagnosis, treatment suggestions, and irrigation recommendations.

The system integrates multiple data sources into a unified decision-support platform, enabling farmers to receive accurate, real-time, and actionable agricultural insights. The proposed approach improves early disease detection, enhances crop management, and supports data-driven farming practices for sustainable agriculture.

Keywords: *Smart Agriculture, Generative Artificial Intelligence, Machine Learning, Internet of Things, Crop Disease Detection, CNN, Large Language Models, Precision Farming.*

TABLE OF CONTENTS

Chapter 1 : INTRODUCTION.....	2
1.1 INTRODUCTION	2
1.2 OBJECTIVES OF PROJECT REPORT.....	3
1.3 ORGANIZATION OF PROJECT REPORT	3
Chapter 2 : LITERATURE SURVEY	6
2.1 LITERATURE SURVEY	6
Chapter 3 : SOFTWARE REQUIREMENTS SPECIFICATIONS	10
3.1 INTRODUCTION	10
3.2 PROBLEM STATEMENT	10
3.3 PROJECT SCOPE	11
3.4 USER CLASSES AND CHARACTERISTICS.....	11
3.5 ASSUMPTIONS AND DEPENDENCIES.....	11
3.5.1 ASSUMPTIONS	11
3.5.2 DEPENDENCIES	11
3.6 FUNCTIONAL REQUIREMENTS	12
3.7 NON-FUNCTIONAL REQUIREMENTS	12
3.7.1 PERFORMANCE REQUIREMENTS	12
3.7.2 SAFETY REQUIREMENTS	12
3.7.3 SECURITY REQUIREMENTS.....	13
3.7.4 SOFTWARE QUALITY ATTRIBUTES	13

3.8	SYSTEM REQUIREMENTS	13
3.8.1	HARDWARE REQUIREMENTS	13
3.8.2	SOFTWARE REQUIREMENTS.....	14
3.9	ANALYSIS MODELS (SDLC MODEL TO BEAPPLIED).....	14
Chapter 4	: SYSTEM DESIGN.....	18
4.1	DATA FLOW DIAGRAM (DFD)	18
4.1.1	DFD LEVEL 0	18
4.1.2	DFD LEVEL 1	18
4.1.3	DFD LEVEL 2	19
4.2	UML DIAGRAMS	20
4.2.1	USE-CASE DIAGRAM	20
4.2.2	CLASS DIAGRAM.....	21
4.2.3	SEQUENCE DIAGRAM.....	22
4.2.4	ACTIVITY DIAGRAM.....	23
4.2.5	COMPONENT DIAGRAM.....	24
Chapter 5	: PROJECT PLAN.....	26
5.1	PROJECT ESTIMATES.....	26
5.1.1	EFFORT ESTIMATE TIMETABLE.....	26
5.1.2	PROJECT DESCRIPTION	27
5.1.3	ESTIMATION OF KLOC	28
5.2	RISK MANAGEMENT.....	28

5.2.1	OVERVIEW OF RISK MITIGATION, MONITORING, MANAGEMENT RISK MANAGEMENT ORGANIZATIONAL ROLE	28
5.3	TIME-LINE CHART	31
Chapter 6 : IMPLEMENTATION.....		33
6.1	SYSTEM ARCHITECTURE.....	33
6.2	ALGORITHMS	34
6.2.1	CONVOLUTIONAL NEURAL NETWORK (CNN).....	34
6.2.2	DECISION TREE ALGORITHM.....	37
6.2.3	GENERATIVE AI (GenAI).....	39
6.2.4	WEATHER API PROCESSING.....	41
6.3	TOOLS AND TECHNOLOGIES USED	43
6.3.1	PYTHON	43
6.3.2	FLASK (WEB FRAMEWORK).....	43
6.3.3	JAVASCRIPT (JS).....	43
6.3.4	HTML & CSS	43
6.3.5	FIREBASE	44
6.3.6	NODEMCU (ESP32).....	44
6.3.7	DHT11 SENSOR.....	44
6.3.8	SOIL MOISTURE SENSOR	44
6.3.9	OPEN-METEO API	44
6.4	CODING	45
6.4.1	LLM SUPPORT	45

6.4.2	DISEASE PREDICTION	Error! Bookmark not defined.
6.5	RESULT SET	49
6.5.1	LOGIN AND REGISTRATION PAGE	49
6.5.2	HOME AND DASHBOARD PAGE.....	50
6.5.3	ADVISORY REPORT.....	52
6.5.4	DISEASE DETECTION.....	55
6.5.5	SENSOR AND API DATA	57
6.5.6	LANGUAGE PAGE.....	57
6.5.7	HOW IT WORKS	58
6.5.8	HARDWARE IMAGE	59
Chapter 7	: TESTING	61
6.1	TEST CASES	61
6.1.1	TEST CASE 1 (LOGIN AND REGISTRATION PAGE).....	61
6.1.2	TEST CASE 2 (LAYOUT).....	62
6.1.3	TEST CASE 3 (HOME PAGE)	63
6.1.4	TEST CASE 4 (DASHBOARD PAGE).....	64
6.1.5	TEST CASE 5 (IMAGE UPLOAD AND AI ANALYSIS PAGES)	65
6.1.6	TEST CASE 6 (SENSOR PAGE).....	67
6.1.7	TEST CASE 7 (LLM SUPPORT OR BOT PAGE).....	69
6.1.8	TEST CASE 8 (REPORT GENERATION PAGE)	70
6.1.9	TEST CASE 9 (LANGUAGE PAGE)	72

6.2	TEST CASES	73
6.2.1	OVERVIEW.....	73
6.2.2	TEST EXECUTION SUMMARY	73
6.2.3	MODULES COVERED	73
6.2.4	ANALYSIS AND INSIGHTS	73
Chapter 8 : CONCLUSION		75
Chapter 9 : REFERENCES.....		77

LIST OF FIGURES

Figure 1. Software Development Lifecycle.....	15
Figure 2. Waterfall Model	15
Figure 3. DFD Level 0	18
Figure 4. DFD Level 1	18
Figure 5. DFD Level 2	19
Figure 6. Use-Case Diagram.....	20
Figure 7. Class Diagram	21
Figure 8. Sequence Diagram.....	22
Figure 9. Activity Diagram	23
Figure 10. Component Diagram	24
Figure 11. Time-Line Chart.....	31
Figure 12. System Architecture	33
Figure 13. Convolutional Neural Network (CNN)	35
Figure 14. Working of CNN	37
Figure 15. Decision Tree Algorithm	37
Figure 16. Decision Tree Algorithm Example.....	38
Figure 17. Generative AI Diagram.....	39
Figure 18. Working of GenAI.....	40
Figure 19. Working of Weather API Processing	42

LIST OF TABLES

Table 1. Effort Estimate Timetable	26
Table 2. Project Schedule	27
Table 3. KLOC of Modules	28

CHAPTER 1

INTRODUCTION

INTRODUCTION

1.1 INTRODUCTION

Agriculture plays a vital role in the global economy and is the primary source of livelihood for a large population, especially in developing countries like India. However, the agricultural sector faces significant challenges due to climate change, unpredictable weather conditions, limited water resources, and increasing crop diseases. These issues directly affect crop yield, farmer income, and food security, making it essential to adopt modern technological solutions.

Traditional farming practices mainly depend on manual observation, experience-based decisions, and conventional methods of irrigation and crop management. While these practices have been effective in the past, they are no longer sufficient to handle the complexities of modern agriculture. Farmers often lack access to timely and accurate information, which leads to inefficient resource utilization and reduced productivity.

With the advancement of technology, modern agriculture has started adopting solutions based on Artificial Intelligence (AI), Internet of Things (IoT), and data analytics. IoT sensors such as soil moisture sensors and temperature-humidity sensors enable real-time monitoring of environmental conditions. Similarly, AI and machine learning models help in analyzing agricultural data and predicting outcomes, allowing farmers to make informed decisions.

One of the major applications of AI in agriculture is plant disease detection using computer vision techniques. Convolutional Neural Networks (CNNs) can analyze leaf images and identify diseases with high accuracy. Early detection of plant diseases helps farmers take preventive measures, reducing crop losses and improving overall yield. However, most existing systems focus only on disease classification and do not provide comprehensive advisory support.

In addition, Generative Artificial Intelligence (GenAI) and Large Language Models (LLMs) have introduced new possibilities in agriculture by generating human-like advisory reports. These models can analyze multiple data sources such as weather data, soil conditions, and crop health to provide clear and actionable recommendations. This helps farmers better understand complex agricultural data and take appropriate actions.

To address the limitations of existing solutions, this research proposes a unified web-based platform called the Farmer Advisory Report Generator. The system integrates real-time weather data, IoT sensor data, and crop disease detection using deep learning techniques. Furthermore, it uses Generative AI to generate detailed advisory reports including treatment suggestions and irrigation recommendations. By combining multiple technologies into a single platform, the proposed system aims to improve decision-making, enhance productivity, and support sustainable agriculture.

1.2 OBJECTIVES OF PROJECT REPORT

The detailed objectives of the project are as follows:

- **To develop a unified web-based platform**

The system aims to integrate multiple agricultural services, including weather forecasting, soil monitoring, and crop disease detection, into a single platform. This eliminates the need for farmers to use multiple applications and simplifies decision-making.

- **To collect real-time environmental data using IoT sensors**

The project focuses on gathering accurate field data using IoT devices such as NodeMCU, soil moisture sensors, and DHT11 temperature and humidity sensors. This data helps in monitoring soil conditions and environmental factors that directly affect crop growth.

- **To implement a CNN-based crop disease detection system**

A Convolutional Neural Network (CNN) model is developed to analyze leaf images uploaded by farmers. The model identifies plant diseases based on visual features, enabling early detection and timely preventive measures.

- **To integrate Generative Artificial Intelligence (GenAI)**

The system incorporates GenAI models to generate natural-language advisory reports. These reports include disease diagnosis, treatment suggestions, irrigation planning, and crop management strategies in an easy-to-understand format.

- **To provide real-time weather forecasting**

The system uses APIs such as Open-Meteo to fetch accurate and up-to-date weather data. This helps farmers plan irrigation, fertilization, and crop protection activities based on weather conditions.

- **To design a user-friendly and accessible interface**

The platform is designed with a simple and intuitive interface that allows farmers to easily upload images, view sensor data, and receive recommendations. The system also supports multilingual features for better accessibility.

- **To improve agricultural productivity and sustainability**

By combining data from multiple sources and providing intelligent recommendations, the system aims to enhance crop yield, optimize resource utilization, and promote sustainable farming practices.

1.3 ORGANIZATION OF PROJECT REPORT

- 1) **Chapter 2** Deals with the Project Related Work i.e., Literature Survey.
- 2) **Chapter 3** Giving an overall view of the techniques used in the system
- 3) **Chapter 4** Deals with System Design.
- 4) **Chapter 5** Project Plan
- 5) **Chapter 6** Implementation Part
- 6) **Chapter 7** Testing
- 7) **Chapter 8** Conclusion
- 8) **Chapter 9** references

CHAPTER 2

LITERATURE SURVEY

LITERATURE SURVEY

2.1 LITERATURE SURVEY

Information systems and digital technologies are increasingly required to provide fast, accurate, and intelligent decision-making solutions in modern agriculture. With the growing challenges such as climate change, unpredictable weather conditions, soil degradation, and crop diseases, traditional farming methods are no longer sufficient. Therefore, technologies like **Artificial Intelligence (AI)**, **Internet of Things (IoT)**, **Machine Learning (ML)**, **Computer Vision**, and **Generative AI** are being widely adopted in agriculture. These technologies enable real-time monitoring, data analysis, and intelligent advisory generation, helping farmers improve productivity and reduce losses.

Machine Learning and IoT in Agriculture:

Machine learning, a major branch of Artificial Intelligence, has been widely applied in agriculture for data analysis and prediction. With the help of IoT devices, large amounts of environmental data such as soil moisture, temperature, and humidity are collected in real time. This data can be analyzed using machine learning algorithms to support smart farming practices. Researchers have developed IoT-based systems that continuously monitor field conditions and assist in irrigation management and resource optimization.

Additionally, machine learning models are used for weather prediction and crop planning. By analyzing historical and real-time data, these models help farmers make better decisions regarding irrigation, fertilization, and harvesting. However, most existing systems focus mainly on data monitoring and prediction without providing complete advisory support.

Decision Tree in Agriculture:

Decision Tree is a widely used supervised machine learning algorithm that is applied for classification and decision-making tasks. It works by dividing data into branches based on specific conditions, making it simple and easy to understand. In agriculture, decision tree algorithms are used to analyze parameters such as soil moisture, temperature, humidity, and weather data.

Researchers have applied decision trees for **crop recommendation**, **irrigation planning**, and **yield prediction**. The model generates clear decision rules, which are easy for farmers to interpret.

Compared to complex machine learning models, decision trees are efficient, require less computational power, and provide faster results. In the proposed system, the decision tree algorithm is used to analyze sensor and weather data to generate smart irrigation and crop management recommendations.

Deep Learning and Crop Disease Detection:

Deep learning techniques, especially **Convolutional Neural Networks (CNNs)**, have shown great success in crop disease detection. CNN models analyze leaf images and identify diseases based on visual features such as color, texture, and shape. These models have achieved high accuracy when trained on agricultural datasets like PlantVillage.

Deep learning enables early detection of plant diseases, allowing farmers to take preventive measures and reduce crop losses. Unlike traditional methods, CNN models automatically extract important features from images. However, most existing systems only perform classification and do not provide detailed treatment recommendations.

Generative AI in Agriculture:

Recent advancements in Artificial Intelligence have introduced **Generative AI and Large Language Models (LLMs)**, which are capable of generating human-like text and intelligent recommendations. These models can analyze multiple data sources such as environmental conditions, crop health, and weather data to generate advisory reports.

Generative AI helps convert complex data into simple and understandable language for farmers. It provides recommendations such as disease treatment, irrigation planning, and crop management strategies. This improves decision-making and reduces dependency on expert consultation. However, integration of Generative AI with IoT and disease detection systems is still limited in existing solutions.

Limitations of Existing Systems:

Although many technologies have been developed for smart agriculture, most existing systems focus on individual functionalities such as IoT monitoring, disease detection, or weather prediction. There is a lack of integrated platforms that combine all these features into a single system.

Some major limitations include:

- Lack of unified advisory systems
- Limited integration of multiple technologies

- Difficulty in interpreting raw data by farmers
- Dependence on multiple platforms for different services

Proposed System Contribution:

To overcome these limitations, the proposed system introduces a **Farmer Advisory Report Generator** that integrates IoT-based environmental monitoring, Decision Tree-based analysis, CNN-based crop disease detection, real-time weather data, and Generative AI-based advisory generation into a single platform.

The system provides accurate, real-time, and easy-to-understand recommendations for farmers, including disease diagnosis, treatment suggestions, and irrigation planning. By combining multiple technologies into one platform, the system improves agricultural productivity, reduces crop losses, and supports intelligent decision-making in modern farming.

CHAPTER 3

SOFTWARE REQUIREMENTS

SPECIFICATION

SOFTWARE REQUIREMENTS SPECIFICATIONS

3.1 INTRODUCTION

This chapter presents the **Software Requirements Specification (SRS)** of the proposed system, *Farmer Advisory Report Generator using Soil, Weather, and Crop Data with Generative Artificial Intelligence (GenAI)*. The purpose of this chapter is to clearly define the functional and non-functional requirements of the system, along with its overall structure and operational behavior.

The system is designed to integrate multiple technologies such as **Internet of Things (IoT), Artificial Intelligence (AI), Computer Vision, and Web Technologies** into a single platform. It collects real-time environmental data from IoT sensors, retrieves weather information through APIs, and analyzes crop leaf images using a Convolutional Neural Network (CNN) model. Based on this data, the system generates intelligent advisory reports to assist farmers in making better decisions.

This chapter also describes the **user requirements, system features, constraints, assumptions, and dependencies** associated with the system. It serves as a foundation for system design, development, and implementation, ensuring that all functionalities are well-defined and aligned with the project objectives.

Overall, the SRS provides a detailed understanding of how the system will function, what features it will include, and how it will meet the needs of farmers by providing accurate, real-time, and user-friendly agricultural guidance.

3.2 PROBLEM STATEMENT

The scope of this project is to develop a smart agricultural advisory system using **Artificial Intelligence (AI), Internet of Things (IoT), Computer Vision, and Generative AI technologies**. The system integrates real-time environmental data collected from IoT sensors such as soil moisture and temperature-humidity sensors, along with weather data obtained through APIs. It also includes a Convolutional Neural Network (CNN) model to detect crop diseases from leaf images uploaded by farmers. Based on the collected data and analysis, the system generates intelligent advisory reports that include disease diagnosis, treatment suggestions, irrigation planning, and crop management strategies, helping farmers make informed and timely decisions.

The project focuses on providing a **user-friendly web-based platform** that allows farmers to easily access environmental data, upload crop images, and receive recommendations in a simple and understandable format. It aims to improve agricultural productivity, optimize resource utilization, and reduce crop losses through data-driven decision support. However, the scope of the project is limited to selected crops and datasets used for training the model, and it depends on the availability of IoT devices and internet connectivity for real-time data processing. Future enhancements may include mobile application support, expansion to more crop varieties, and integration of advanced analytics for predictive farming.

3.3 PROJECT SCOPE

- The scope of this project is to develop a smart agricultural advisory system using Artificial Intelligence (AI), Internet of Things (IoT), and Generative AI technologies.
- This project ensures accurate monitoring of environmental conditions, crop disease detection, and generation of advisory reports for farmers using real-time data and intelligent models.
- The scope of this project is limited to a web-based platform that supports selected crops and depends on IoT devices and internet connectivity for proper functioning.

3.4 USER CLASSES AND CHARACTERISTICS

The system is designed for **farmers, administrators, and developers**. Farmers are primary users with basic technical knowledge, while administrators manage system data and monitoring. Developers handle system maintenance, updates, and improvements.

3.5 ASSUMPTIONS AND DEPENDENCIES

3.5.1 ASSUMPTIONS

1. Users have basic knowledge of smartphones or web applications.
2. Internet connectivity is available.
3. IoT sensors are properly installed and working.
4. Users can upload crop images correctly.
5. The system is used under normal operating conditions.

3.5.2 DEPENDENCIES

1. Weather API (Open-Meteo) for real-time data.
2. IoT hardware (NodeMCU, soil moisture sensor, DHT11).
3. AI/ML models for disease detection and advisory generation.

4. Internet connectivity for data fetching and processing.
5. Web browser compatibility for accessing the system.
6. Well Trained dataset.

3.6 FUNCTIONAL REQUIREMENTS

The system allows users to register and log in securely, upload crop leaf images for disease detection using a CNN model, and view real-time environmental data such as soil moisture, temperature, and humidity collected from IoT sensors. It also fetches weather information through APIs and generates advisory reports using Generative AI, including disease diagnosis, treatment suggestions, and irrigation recommendations. Additionally, users can access previous reports, and the admin can manage system data and monitor performance.

3.7 NON-FUNCTIONAL REQUIREMENTS

3.7.1 PERFORMANCE REQUIREMENTS

- The system should respond quickly during image upload, disease detection, and report generation without delays.
- Any error occurring in the system should display a clear and understandable error message to the user.
- The system should handle real-time data from IoT sensors and weather APIs efficiently without confusion or interruption.
- The performance of the system should remain stable and efficient even when multiple users access it simultaneously.

3.7.2 SAFETY REQUIREMENTS

The system should ensure safe and reliable operation by handling errors properly and preventing system crashes during execution. It must protect user data from loss or corruption and ensure that all processes run smoothly without causing unexpected failures. The system should also provide proper validation of inputs, such as uploaded images and sensor data, to avoid incorrect processing. Additionally, it should maintain data consistency and ensure that any failure or interruption does not affect the overall functionality of the system.

3.7.3 SECURITY REQUIREMENTS

The system should ensure proper security by implementing user authentication and authorization to prevent unauthorized access. It must protect user data, including personal information and uploaded images, from misuse or unauthorized modifications. Secure communication protocols should be used to safeguard data transmission between the system and external services such as APIs. Additionally, the system should include measures to prevent common security threats such as data breaches, unauthorized login attempts, and malicious activities.

3.7.4 SOFTWARE QUALITY ATTRIBUTES

1. **Usability:** The system should have a simple and user-friendly interface so that farmers with minimal technical knowledge can easily operate it without confusion.
2. **Reliability:** The system should provide accurate disease detection results and consistent performance without frequent failures or errors.
3. **Scalability:** The system should be capable of handling an increasing number of users, data, and system load without affecting performance.
4. **Maintainability:** The system should be easy to maintain, update, and modify, allowing developers to fix issues and add new features efficiently.
5. **Efficiency:** The system should utilize resources effectively and provide fast processing for data analysis and report generation.
6. **Availability:** The system should be accessible whenever required with minimal downtime, ensuring continuous service for users.

3.8 SYSTEM REQUIREMENTS

3.8.1 HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or higher
- Speed: 2.0 GHz or higher
- RAM: 4 GB (min)
- Hard Disk: 20 GB (min)
- IoT Device: NodeMCU (ESP8266)
- Sensors: Soil Moisture Sensor, DHT11 Sensor

3.8.2 SOFTWARE REQUIREMENTS

- Operating System: Windows/Linux
- Programming Language: Python
- Web Framework: Flask
- Database: MongoDB/Firebase
- Frontend: HTML, CSS, JavaScript
- AI/ML Libraries: TensorFlow, Keras
- Tools: VS Code
- Web Browser: Chrome / Edge / Firefox

3.9 ANALYSIS MODELS (SDLC MODEL TO BE APPLIED)

Software Development Life Cycle (SDLC) is a structured process used to design, develop, and test high-quality software. It ensures that the software meets user requirements, is completed within time and cost constraints, and maintains good performance and reliability. SDLC provides a clear framework that defines all the stages involved in software development, helping in systematic planning, execution, and maintenance of the system.

- SDLC is the acronym of Software Development Life Cycle.
- SDLC stands for Software Development Life Cycle.
- It is also known as the Software Development Process.
- It defines a set of activities performed at each stage of software development.
- It helps in improving software quality and overall development efficiency.

For this project, the **Waterfall SDLC model** is applied. In this model, development is carried out in a sequential manner where each phase is completed before moving to the next. The phases include requirement analysis, system design, implementation, testing, deployment, and maintenance. This model is suitable for the project as the requirements are clearly defined, and it ensures a structured and systematic development process.

The following figure is a graphical representation of the various stages of a typical SDLC.

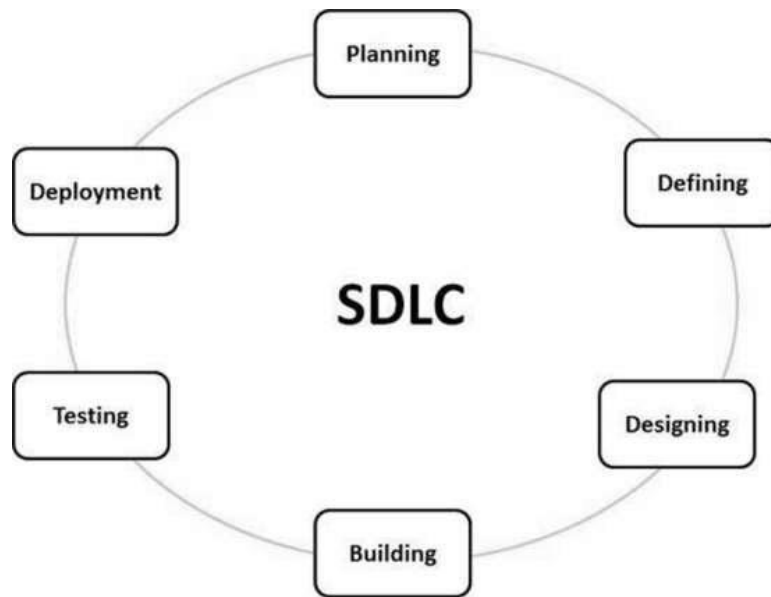


Figure 1. Software Development Lifecycle

Waterfall Model

Waterfall approach was first SDLC Model to be used widely in Software Engineering to ensure success of the project. In "The Waterfall" approach, the whole process of software development is divided into separate phases. In this Waterfall model, typically, the outcome of one phase acts as the input for the next phase sequentially.

The following illustration is a representation of the different phases of the Waterfall Model.

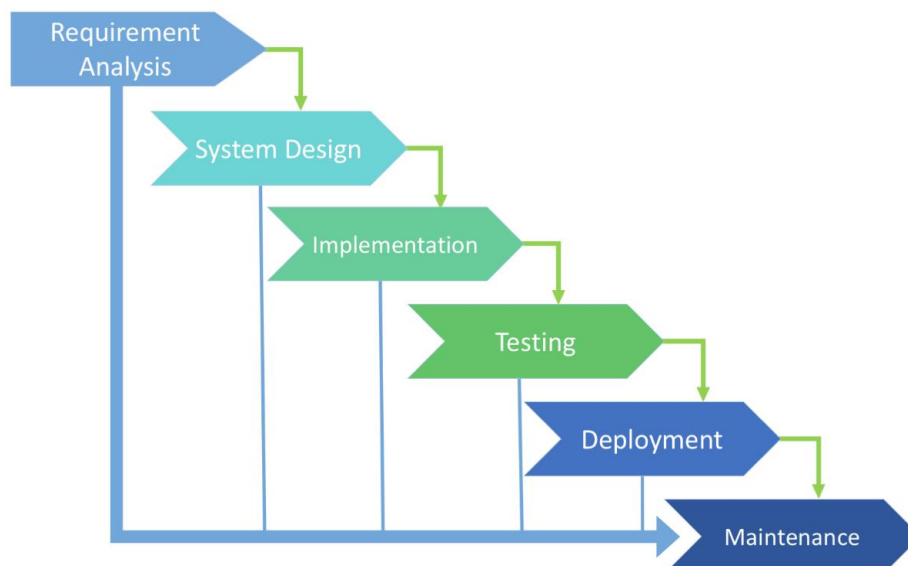


Figure 2. Waterfall Model

The sequential phases in Waterfall model are –

1. **Requirement Gathering and analysis** – All possible requirements of the system to be developed are captured in this phase and documented in a requirement specification document.
2. **System Design** – The requirement specifications from first phase are studied in this phase and the system design is prepared. This system design helps in specifying hardware and system requirements and helps in defining the overall system architecture.
3. **Implementation** – With inputs from the system design, the system is first developed in small programs called units, which are integrated in the next phase. Each unit is developed and tested for its functionality, which is referred to as Unit Testing.
4. **Integration and Testing** – All the units developed in the implementation phase are integrated into a system after testing of each unit. Post integration the entire system is tested for any faults and failures.
5. **Deployment of system** – Once the functional and non-functional testing is done; the product is deployed in the customer environment or released into the market.
6. **Maintenance** – There are some issues which come up in the client environment. To fix those issues, patches are released. Also, to enhance the product some better versions are released. Maintenance is done to deliver these changes in the customer environment.

CHAPTER 4

SYSTEM DESIGN

SYSTEM DESIGN

4.1 DATA FLOW DIAGRAM (DFD)

4.1.1 DFD LEVEL 0

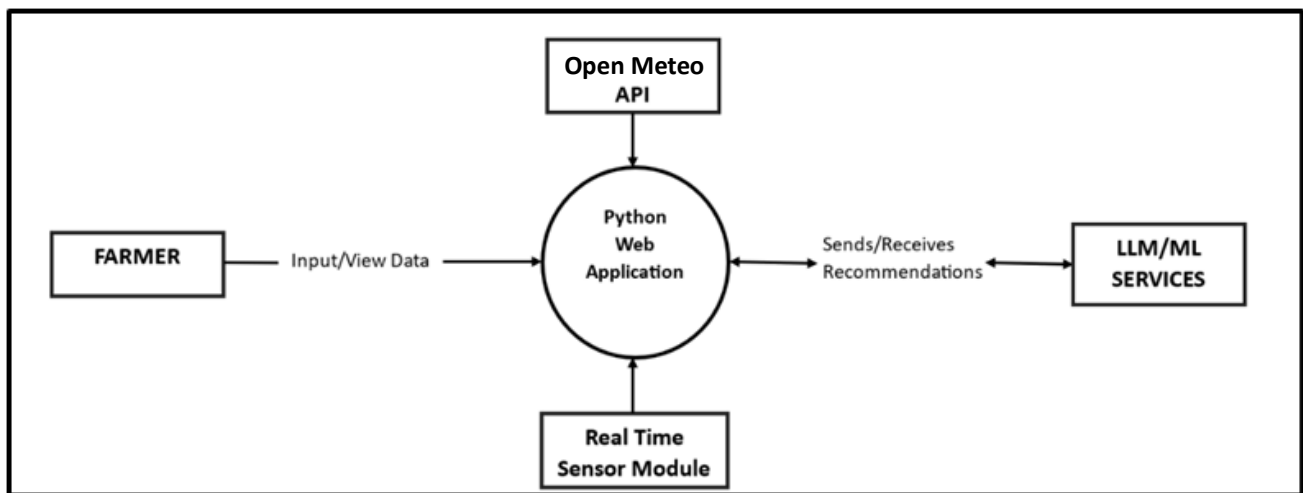


Figure 3. DFD Level 0

4.1.2 DFD LEVEL 1

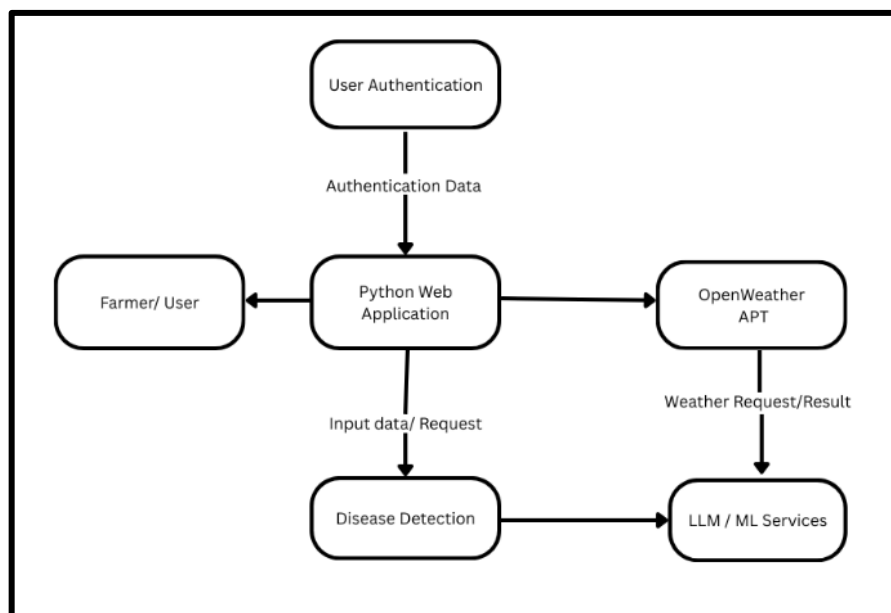


Figure 4. DFD Level 1

4.1.3 DFD LEVEL 2

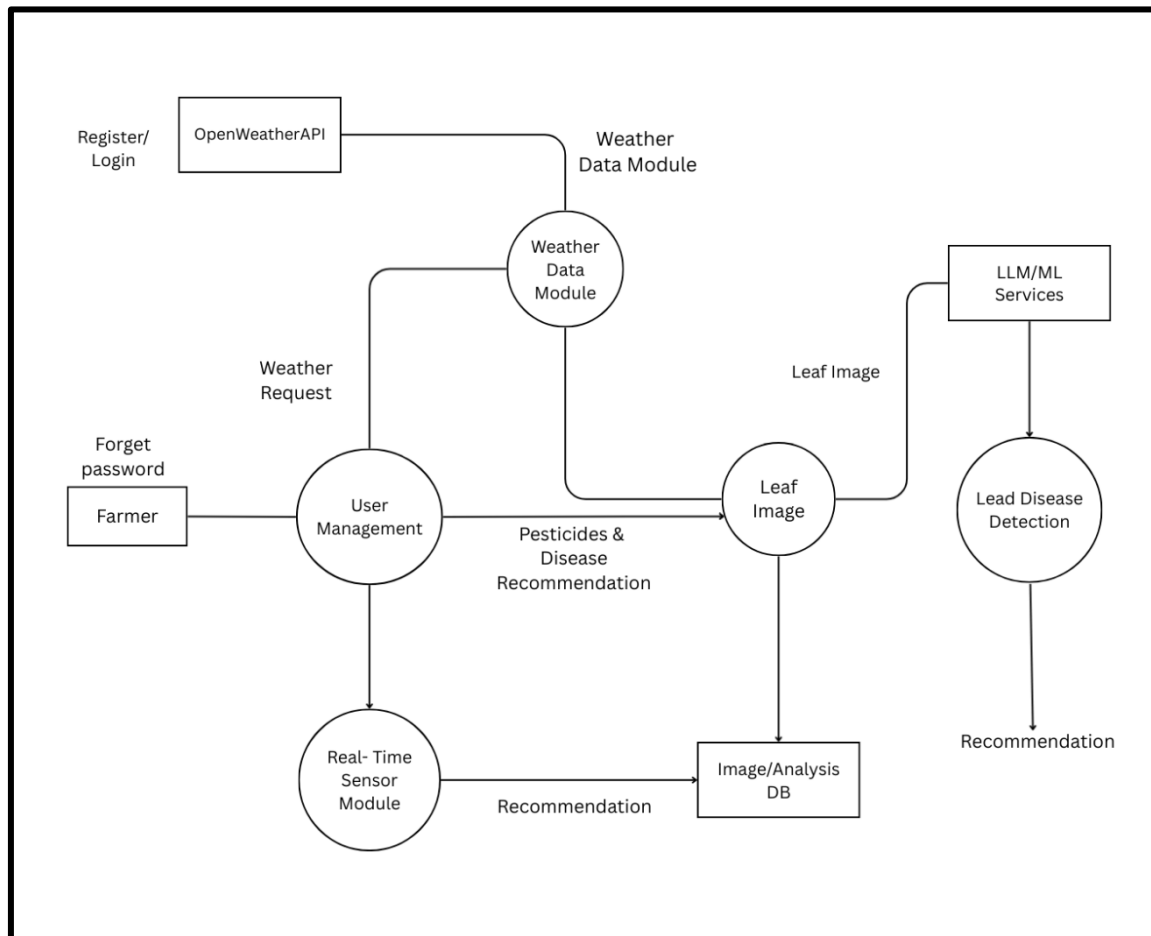


Figure 5. DFD Level 2

4.2 UML DIAGRAMS

4.2.1 USE-CASE DIAGRAM

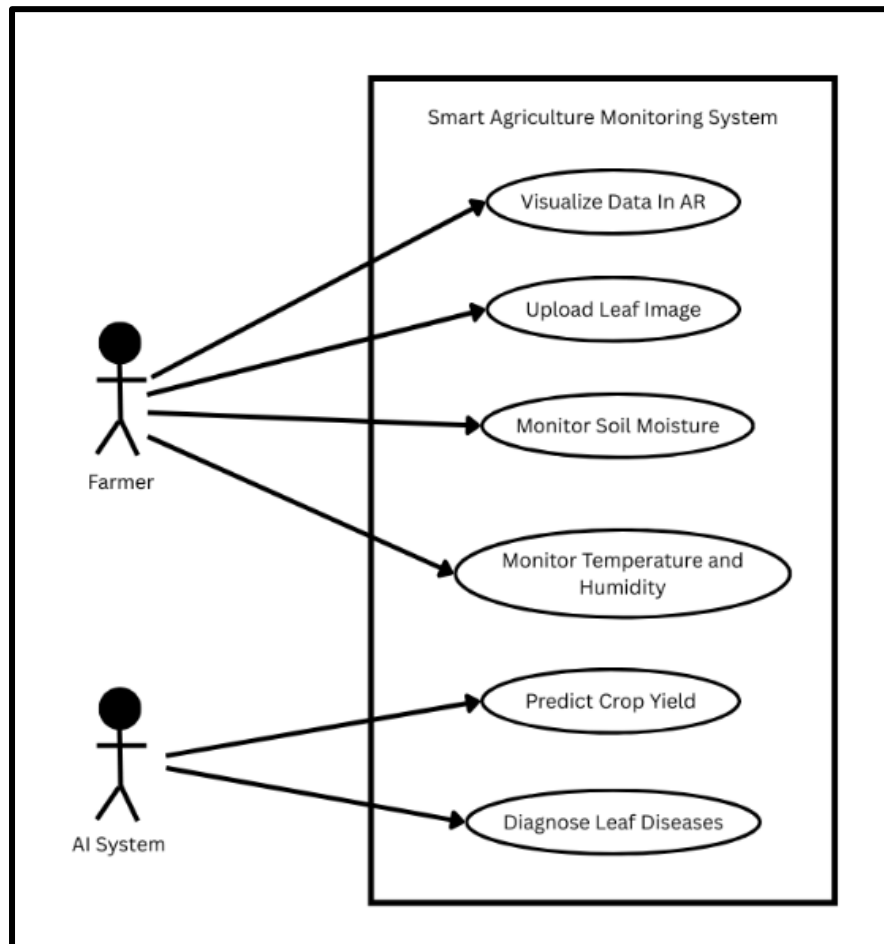


Figure 6. Use-Case Diagram

4.2.2 CLASS DIAGRAM

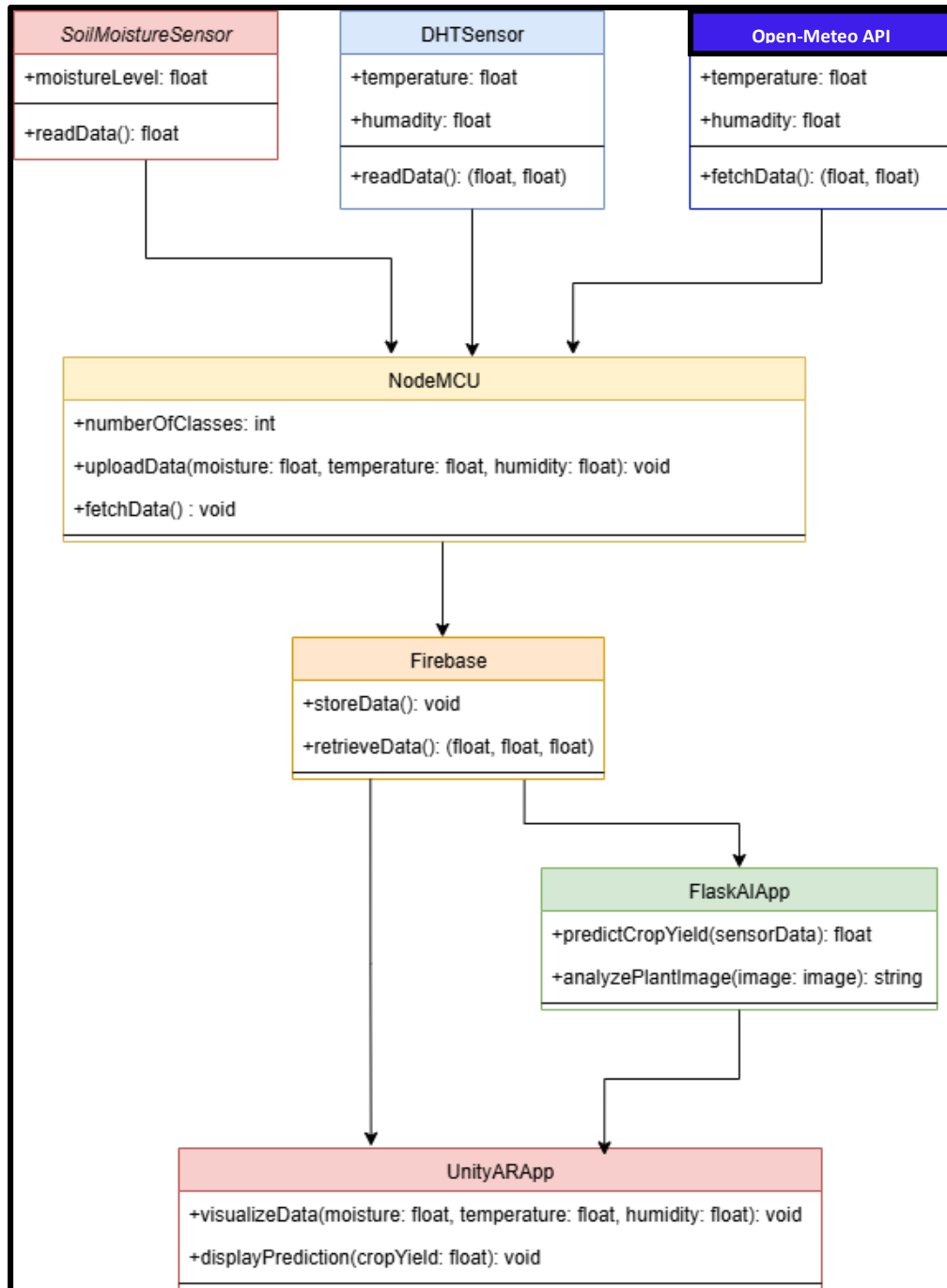


Figure 7. Class Diagram

4.2.3 SEQUENCE DIAGRAM

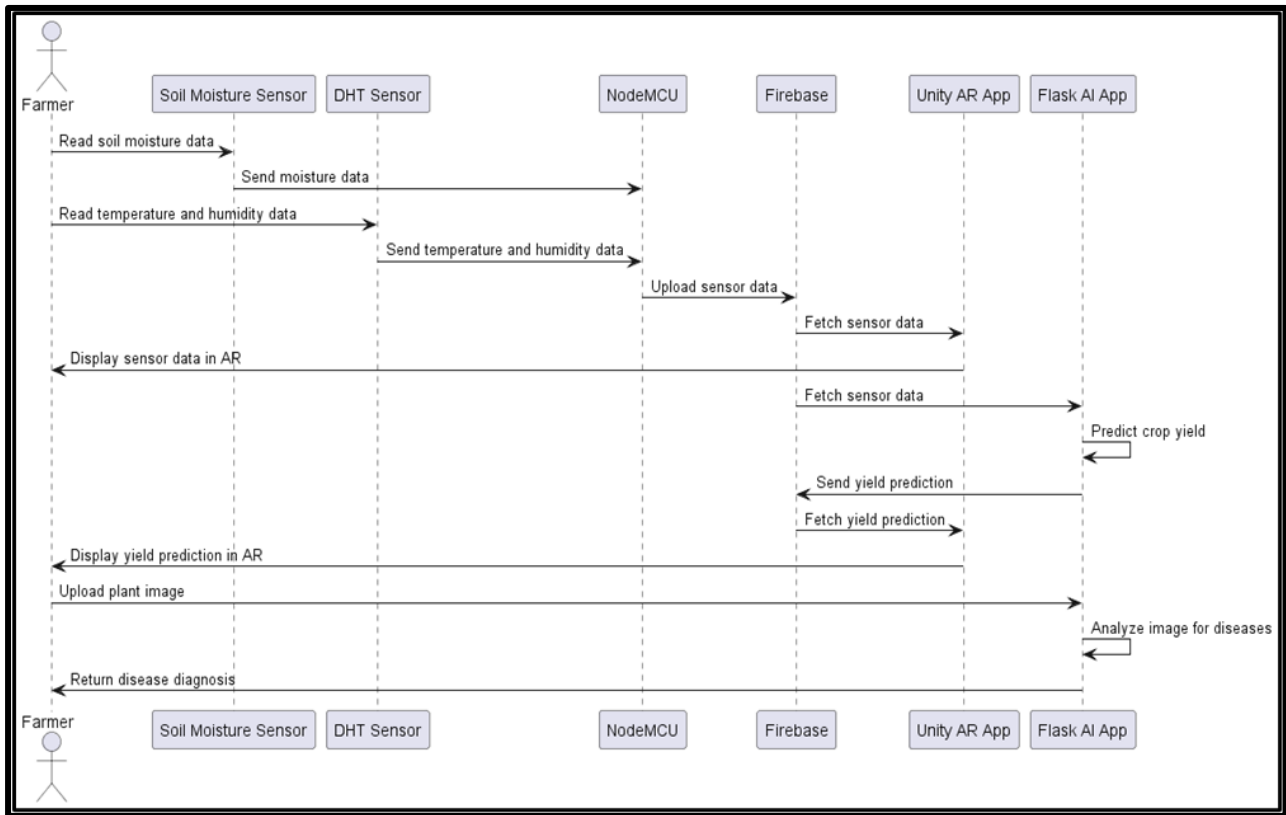


Figure 8. Sequence Diagram

4.2.4 ACTIVITY DIAGRAM

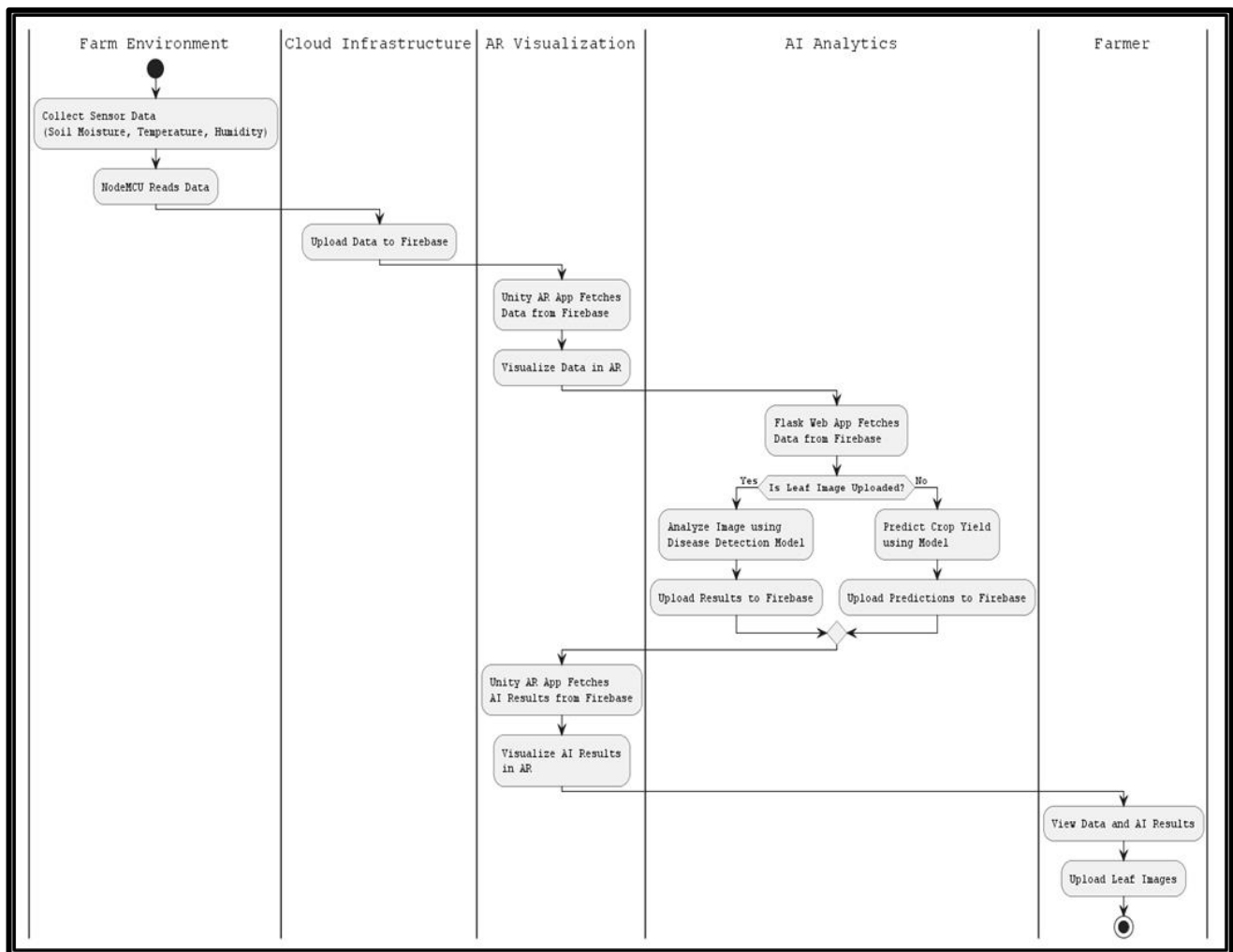


Figure 9. Activity Diagram

4.2.5 COMPONENT DIAGRAM

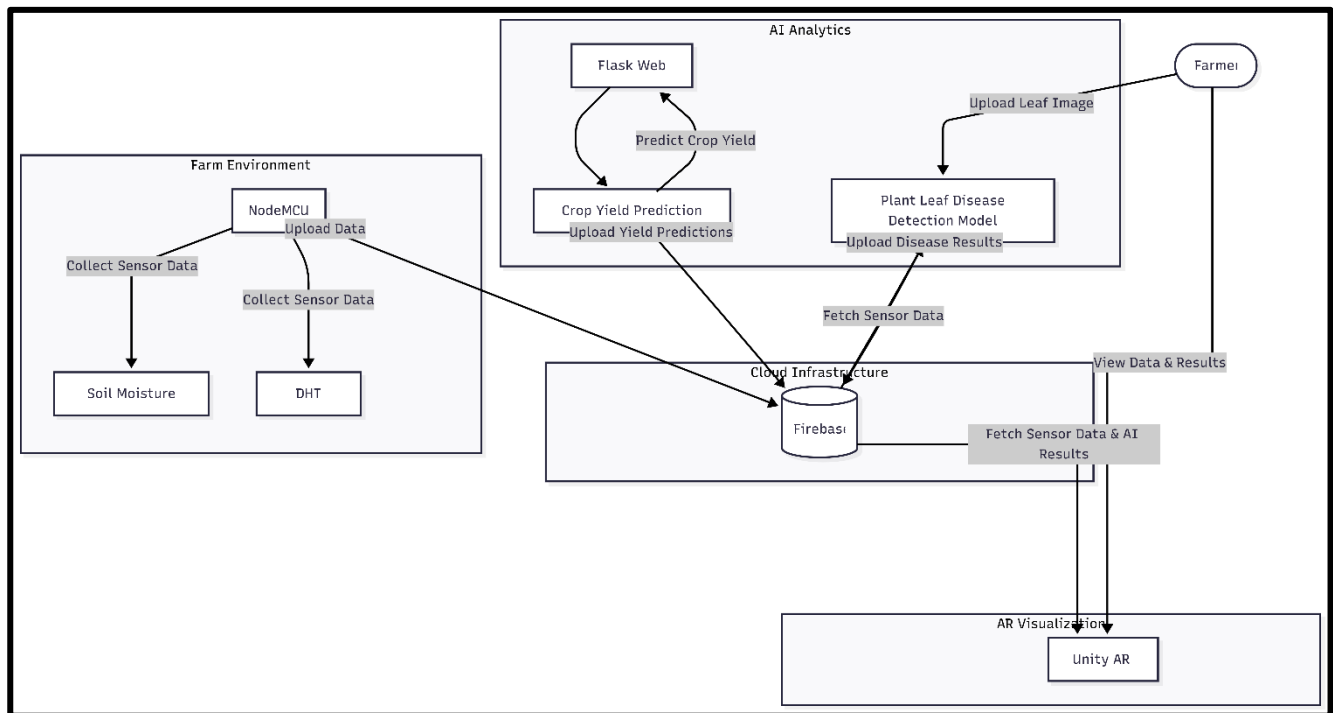


Figure 10. Component Diagram

CHAPTER 5

PROJECT PLAN

PROJECT PLAN

5.1 PROJECT ESTIMATES

5.1.1 EFFORT ESTIMATE TIMETABLE

TASK	EFFORT WEEKS	DELIVERABLES	MILESTONES
Analysis of existing system & comparison with proposed system	2 weeks	Requirement Analysis Report	-
Literature survey	1 week	Literature Document	-
Designing & planning	2 weeks	Design Document	-
System flow	1 week	DFD & UML Diagrams	-
Designing modules & its deliverables	2 weeks	Module Design Document	-
Implementation (IoT + CNN + Decision Tree + GenAI)	6 weeks	Primary System	-
Testing	3 weeks	Test Reports	Formal
Documentation	2 weeks	Final Project Report	Formal

Table 1. Effort Estimate Timetable

5.1.2 PROJECT DESCRIPTION

PHASE	TASK	DESCRIPTION
Phase 1	Analysis	Analyse the problem related to agriculture and study requirements of IoT, AI, and advisory systems.
Phase 2	Literature survey	Collect and study research papers related to IoT, CNN, Decision Tree, and Generative AI in agriculture.
Phase 3	Design	Design system architecture, modules, and process flow including sensor data, disease detection, and advisory system.
Phase 4	Implementation	Develop IoT setup, implement CNN for disease detection, Decision Tree for recommendations, and integrate all modules.
Phase 5	Testing	Test all modules and overall system to ensure proper working and accuracy of results.
Phase 6	Documentation	Prepare complete project report including results, conclusion, and future scope.

Table 2. Project Schedule

5.1.3 ESTIMATION OF KLOC

SR.NO.	MODULES	KLOC
1	User Authentication & Web Interface	2.00
2	IoT Sensor Data Module (NodeMCU + Sensors)	1.20
3	Disease Detection Module (CNN)	1.80
4	Recommendation Module (Decision Tree + GenAI (LLM))	1.50
5	Weather API Integration	0.50
TOTAL:		7.00

Table 3. KLOC of Modules

5.2 RISK MANAGEMENT

5.2.1 OVERVIEW OF RISK MITIGATION, MONITORING, MANAGEMENT RISK MANAGEMENT ORGANIZATIONAL ROLE

Each team member is responsible for risk management throughout the project. The development team continuously monitors project progress to identify possible risks at early stages. Regular discussions and reviews are conducted to ensure that any issues related to system performance, IoT hardware setup, data transmission, or model accuracy are addressed promptly. All team members actively contribute to identifying and managing risks, including issues related to sensor reliability, system integration, and AI model performance. Proper mitigation strategies are applied to resolve these risks and ensure smooth development and successful completion of the project.

BUSINESS IMPACT RISK

Amount and quality of documentation that must be produced and delivered to the customer: The customer (project guide/college) will be provided with complete project documentation including SRS, design documents, implementation details, and user manual for the application. The customer will also have access to all development-related documents as part of project evaluation.

- **Governmental constraints in the construction of the product:** None known.
- **Costs associated with late delivery:** Late submission of the project may lead to incomplete evaluation and may affect the academic grading of all team members.
- **Costs associated with a defective product:** A defective system may result in incorrect disease detection or wrong advisory recommendations, reducing the reliability and effectiveness of the application.

CUSTOMER RELATED RISKS

- **Have you worked with the customer in the past?** Yes, all team members have completed previous academic projects under the guidance of the same project guide, although this project is more advanced and involves multiple technologies like IoT and AI.
- **Does the customer have a solid idea of what is required?** Yes, the requirements are clearly defined through the System Requirements Specification (SRS) and project guidelines provided by the guide.
- **Will the customer agree to spend time in formal requirements gathering meetings to identify project scope?** Limited interaction is expected. The project guide will provide feedback during reviews, but formal requirement meetings may not be conducted frequently.

PROCESS RISKS

- **Does senior management support a written policy statement that emphasizes the importance of a standard process for software development?** Yes, the project follows standard academic guidelines and a defined SDLC model to ensure systematic development.
- **Has your organization developed a written description of the software process to be used on this project?** Yes, the software development process is clearly defined in the project documentation and follows a structured approach.
- **Are staff members willing to use the software process?** Yes, all team members have agreed to follow the defined development process before starting the project.
- **Is the software process used for other products?** Yes, similar development processes are used in previous academic projects and are suitable for this system.

TECHNICAL ISSUES

- **Are facilitated application specification techniques used to aid in communication between the customer and the developer?**

The development team conducts regular discussions with the project guide to review progress and gather feedback. These interactions are mostly informal, where system features, performance, and improvements are discussed and noted for further development.

- **Are specific methods used for software analysis?**

Yes, various methods such as testing, debugging, and performance evaluation are used to analyze the system. The software is tested for accuracy (disease detection), data consistency (sensor data), and overall system performance to ensure reliability.

- **Do you use a specific method for data and architectural design?**

Yes, the system follows a modular and object-oriented design approach. This ensures better data management, code reusability, and easy integration of components such as IoT modules, machine learning models, and web application.

TECHNOLOGY RISK

- **Is the technology to be built new to your organization?**

No, the technologies used such as Python, IoT (NodeMCU), CNN, Decision Tree, and web development are known and previously studied by the team members.

- **Does the software interface with new or unproven hardware?**

No, the system uses standard and widely used hardware components such as NodeMCU, DHT11 sensor, and soil moisture sensor.

- **Is a specialized user interface demanded by the product requirements?**

Yes, the system requires a simple, user-friendly web interface that is easy to use for farmers with minimal technical knowledge.

DEVELOPMENT ENVIRONMENT RISKS

No advanced project management tools are used in this project due to time constraints, and development is carried out using basic tools like Python and VS Code. Project progress is managed through regular team discussions. Although this approach works for this project, it may lead to minor difficulties in tracking tasks and coordination. However, the team ensures smooth development through continuous communication and proper task distribution. In future, the use of advanced project management and collaboration tools can further improve efficiency, monitoring, and overall project organization.

5.3 TIME-LINE CHART

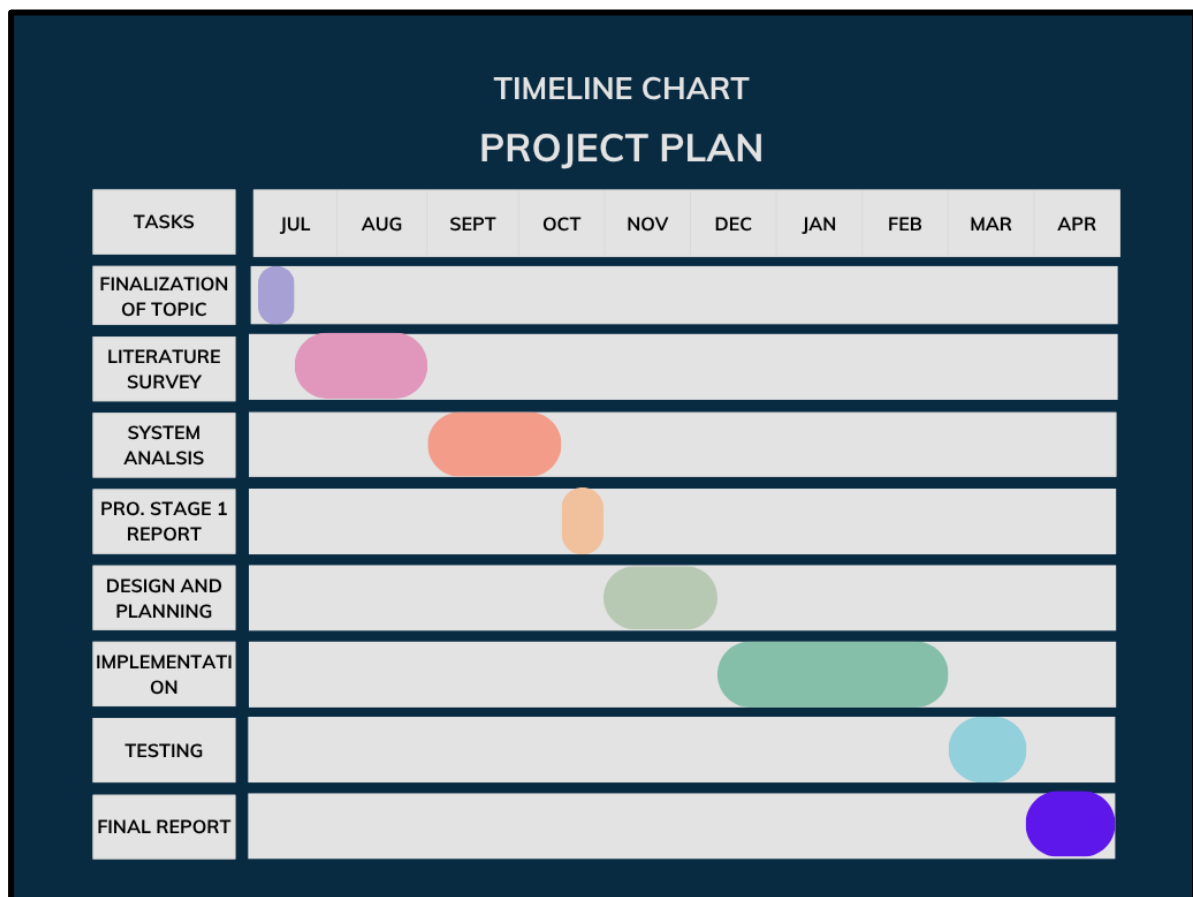


Figure 11. Time-Line Chart

CHAPTER 6

IMPLEMENTATION

IMPLEMENTATION

6.1 SYSTEM ARCHITECTURE

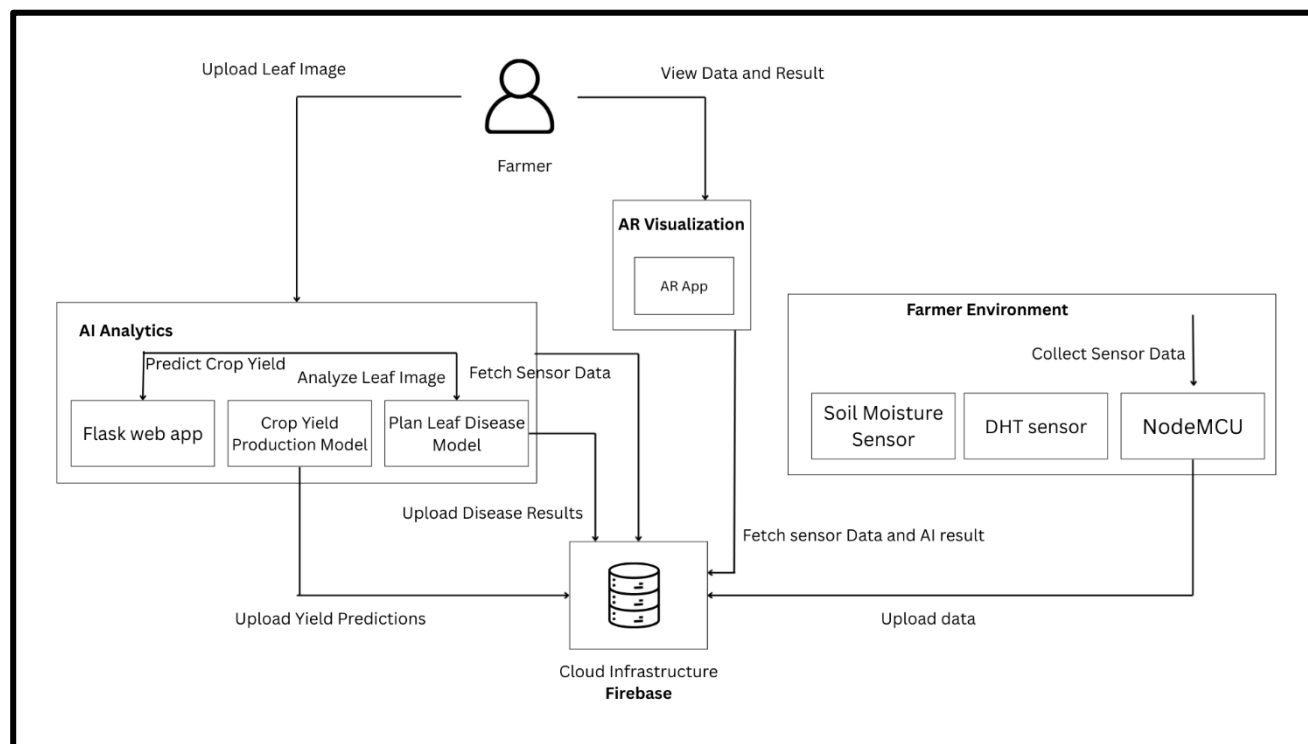


Figure 12. System Architecture

The breakdown of the System in modules is listed below along with their functionalities: -

1) Module 1: User Interface Module

In this module, a web-based interface is developed which allows farmers to interact with the system. The user can register/login, upload leaf images for disease detection, and view sensor data and advisory reports. The interface is designed to be simple and user-friendly so that farmers with minimal technical knowledge can easily use the system.

2) Module 2: IoT Sensor Data Module

This module is responsible for collecting real-time environmental data from the field. It uses IoT devices such as NodeMCU, soil moisture sensor, and DHT sensor to measure parameters like soil moisture, temperature, and humidity. The collected data is transmitted to the cloud (Firebase) for storage and further processing.

3) **Module 3: Weather API Module**

This module collects real-time weather data using APIs such as Open-Meteo. It fetches information like temperature, humidity, rainfall, and weather conditions based on location. This data is used to support better decision-making for irrigation and crop management.

4) **Module 4: Disease Detection Module**

The Disease Detection module is used to identify plant diseases from leaf images uploaded by the farmer. It uses a Convolutional Neural Network (CNN) model to analyze image features such as color, texture, and patterns. The model predicts the type of disease and provides accurate results, helping farmers take early preventive measures.

5) **Module 5: Recommendation & Advisory Module**

This module generates intelligent recommendations based on sensor data and disease detection results. It uses Decision Tree and Generative AI to provide suggestions such as irrigation planning, treatment methods, and crop management strategies. The advisory is generated in simple language for better understanding.

6) **Module 6: Cloud Storage Module**

This module uses Firebase to store all system data including sensor readings, disease detection results, and advisory reports. It acts as a centralized database that enables real-time data access and synchronization between different modules.

6.2 ALGORITHMS

6.2.1 CONVOLUTIONAL NEURAL NETWORK (CNN)

- Convolutional Neural Network (CNN) is a type of deep learning algorithm that is specially designed for processing and analyzing visual data such as images. It is a class of Artificial Neural Network (ANN) that automatically learns important features from input images without the need for manual feature extraction.
- CNN is widely used in applications such as image classification, object detection, and pattern recognition. In this project, CNN is used for **plant disease detection**, where farmers upload leaf images, and the model identifies diseases based on visual characteristics.

- The strength of CNN lies in its ability to detect complex patterns such as colour variations, textures, and shapes in images. This makes it highly suitable for agricultural applications where disease symptoms appear in different forms on leaves.

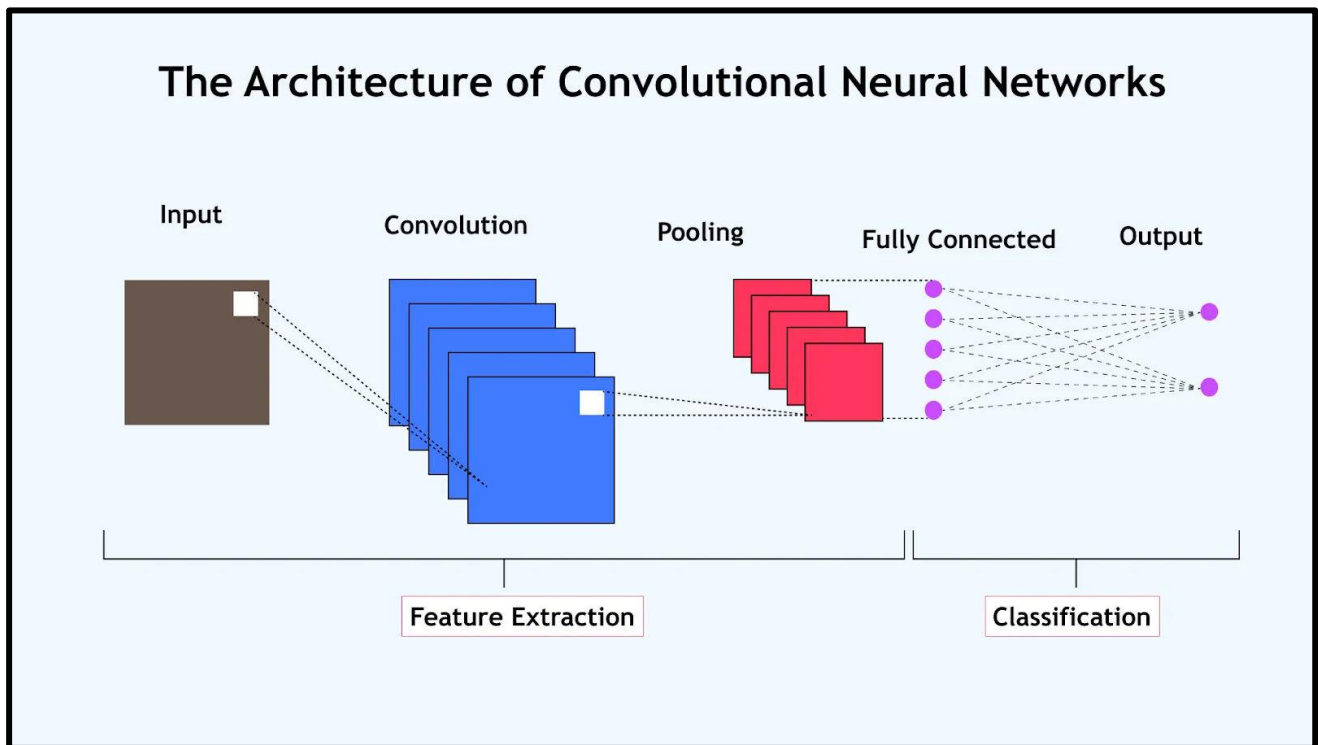


Figure 13. CNN Diagram

➤ Key elements of CNN

1. Input Layer:

The input layer receives the image data in the form of pixel values. For example, a leaf image is converted into a matrix of pixel values (RGB or grayscale).

2. Convolution Layer:

This layer applies multiple filters (kernels) over the input image to extract important features such as edges, textures, and patterns. Each filter produces a feature map that highlights specific characteristics of the image.

3. Activation Function (ReLU):

After convolution, an activation function like ReLU (Rectified Linear Unit) is applied to introduce non-linearity. It converts negative values to zero and keeps positive values, helping the network learn complex patterns.

4. Pooling Layer:

Pooling reduces the size of feature maps while preserving important information. Common types include Max Pooling and Average Pooling. This helps in reducing computational complexity and overfitting.

5. Flattening Layer:

The 2D feature maps obtained from previous layers are converted into a 1D vector so that they can be processed by the fully connected layer.

6. Fully Connected Layer:

This layer connects all neurons and performs classification based on extracted features. It combines all learned features to make a decision.

7. Output Layer:

The final layer provides the classification result, such as identifying whether the leaf is healthy or affected by a specific disease.

➤ How CNN works

1. Image Input:

The system takes a plant leaf image uploaded by the user as input.

2. Preprocessing:

The image is resized, normalized, and converted into a suitable format for processing.

3. Feature Extraction:

Convolution layers apply filters to detect edges, textures, and disease patterns in the image.

4. Feature Reduction:

Pooling layers reduce the image size while preserving important features.

5. Classification:

Fully connected layers analyze the extracted features and classify the image into different disease categories.

6. Prediction Output:

The model outputs the predicted disease type along with confidence level.

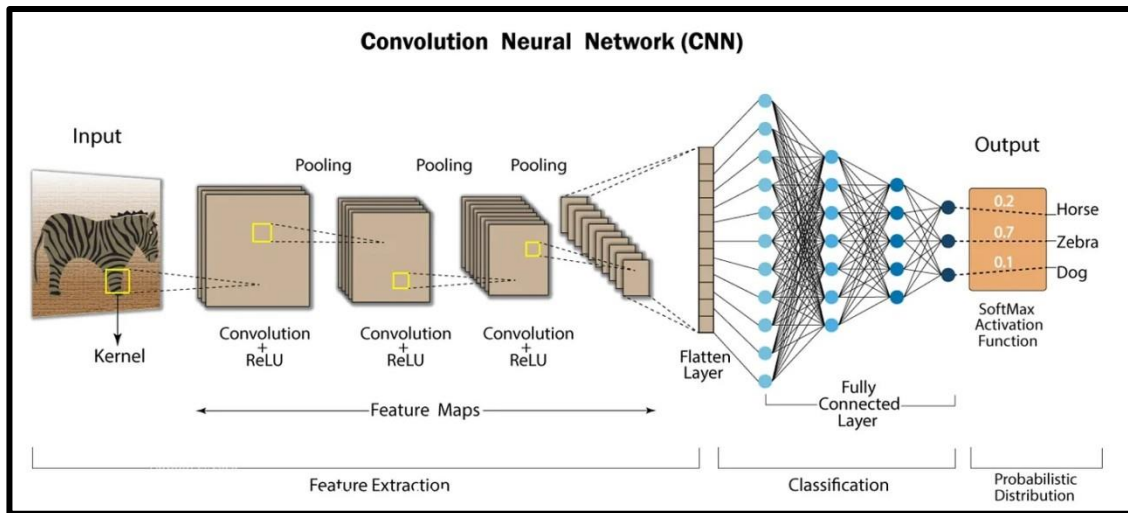


Figure 14. Working of CNN

6.2.2 DECISION TREE ALGORITHM

A Decision Tree Algorithm is a supervised machine learning algorithm used for classification and decision-making problems. It works by creating a tree-like structure where each internal node represents a condition or test on an attribute, each branch represents the outcome of the test, and each leaf node represents the final decision or output. Decision Tree is simple, easy to understand, and widely used in real-time systems.

In this project, the Decision Tree algorithm is used to analyze environmental data such as soil moisture, temperature, and humidity to generate recommendations for irrigation and crop management. It helps farmers make better decisions based on real-time conditions.

Working of Decision Tree Algorithm

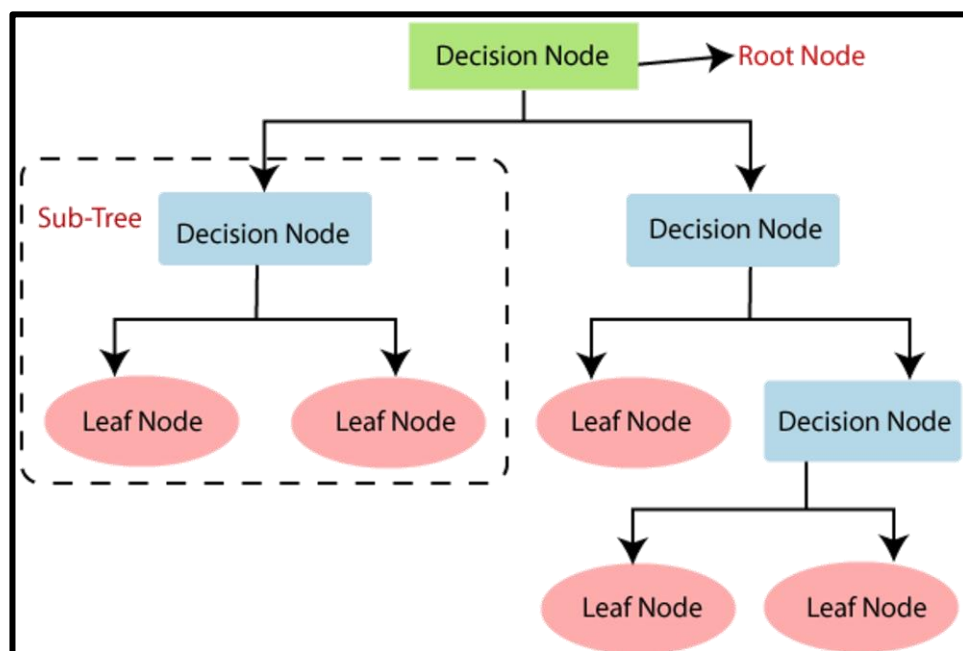


Figure 15. Decision Tree Algorithm

Step-1: Collect input data such as soil moisture, temperature, and humidity.

Step-2: Select the best attribute (feature) to split the data based on conditions.

Step-3: Create a decision node and divide the dataset into subsets.

Step-4: Repeat the process for each subset until all data is classified.

Step-5: Generate final decision rules for prediction.

Example:

Suppose the system receives input data from sensors:

- Soil Moisture = Low
- Temperature = High

The Decision Tree will analyze these conditions and may generate a rule such as:

☞ *If soil moisture is low → Recommend irrigation*

Similarly, different combinations of inputs will produce different recommendations for farmers.

The working of the algorithm can be better understood by the below example:

Example: The diagram represents how a **Decision Tree algorithm** works for classification. It starts with input features such as age, gender, and occupation. The tree begins with a root condition (e.g., $Age < 15$), which splits the data into different branches based on outcomes (Yes/No). Each branch leads to another decision node (e.g., *Is male?*), further dividing the data until it reaches the leaf nodes. These leaf nodes represent the final prediction or decision, such as whether a person likes computer games, along with a prediction score. This hierarchical structure helps in making simple and clear decisions based on input conditions.

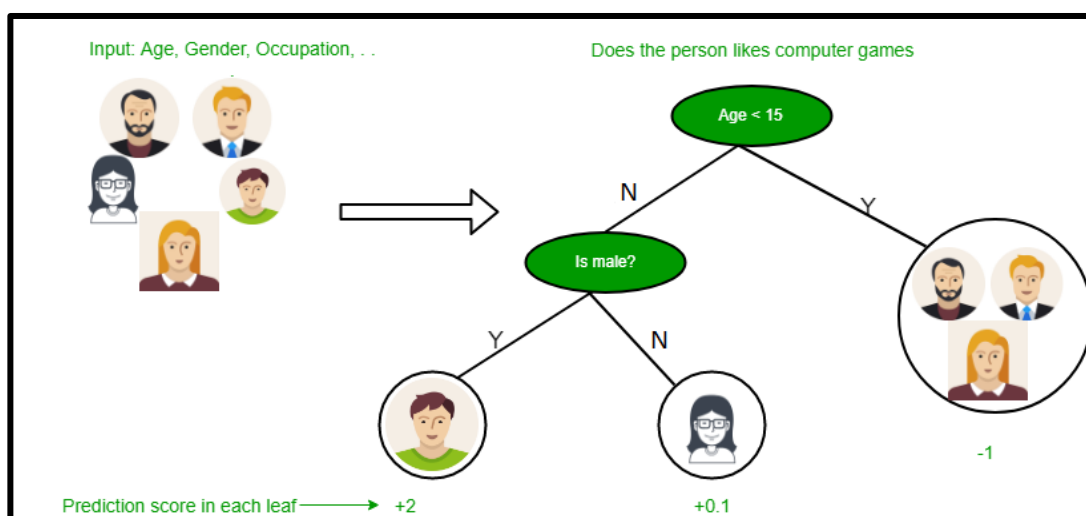


Figure 16. Decision Tree Algorithm Example

6.2.3 Generative AI

- Generative Artificial Intelligence (GenAI) is an advanced AI technology that is capable of generating human-like text, recommendations, and insights based on input data. It uses Large Language Models (LLMs) to analyze data and provide meaningful outputs in natural language.
- In this project, GenAI is used to generate advisory reports for farmers based on multiple inputs such as sensor data, weather data, and disease detection results. It helps in converting technical data into simple and understandable suggestions.
- GenAI improves decision-making by providing real-time recommendations such as irrigation planning, disease treatment, and crop management strategies, making the system more intelligent and user-friendly.

➤ Key elements of a Generative AI

1. Input Data Processing:

GenAI takes multiple inputs such as sensor data (temperature, humidity, soil moisture), weather data, and disease detection results.

2. Context Understanding:

The model analyzes the relationship between different inputs to understand the current agricultural condition.

3. Natural Language Generation:

Based on the analysis, GenAI generates human-readable advisory reports in simple language.

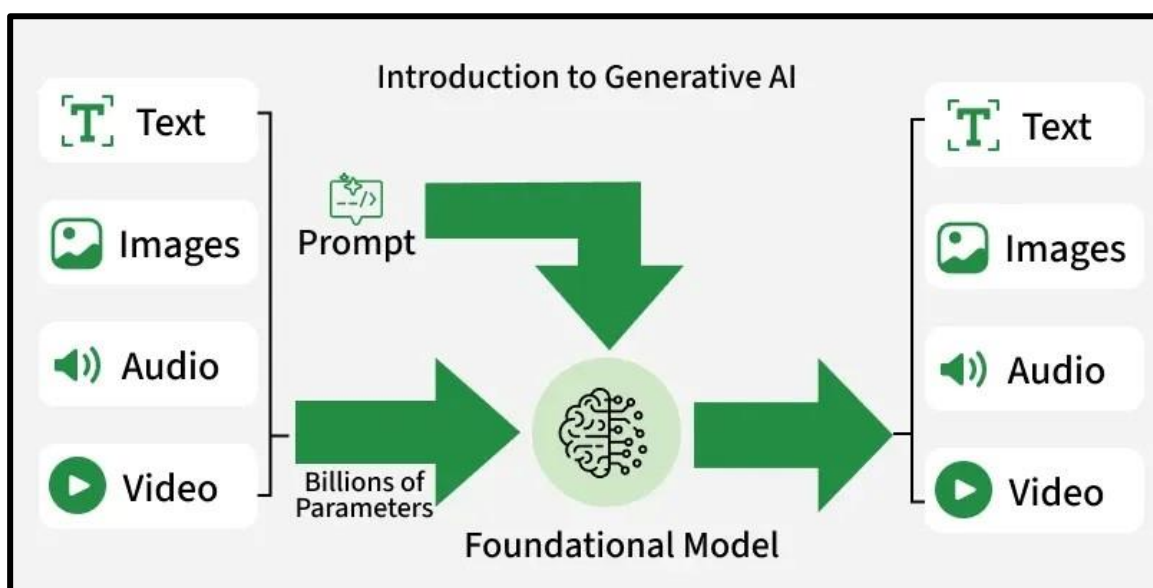


Figure 17. Generative AI Diagram

➤ How Generative AI works

1. Data Collection:

The system collects data from IoT sensors, weather API, and disease detection model.

2. Data Analysis:

The collected data is processed and combined to understand the current condition of crops and environment.

3. Advisory Generation:

GenAI generates recommendations such as irrigation advice, disease treatment, and crop care strategies.

4. Output Delivery:

The generated advisory is displayed to the farmer through the web application.

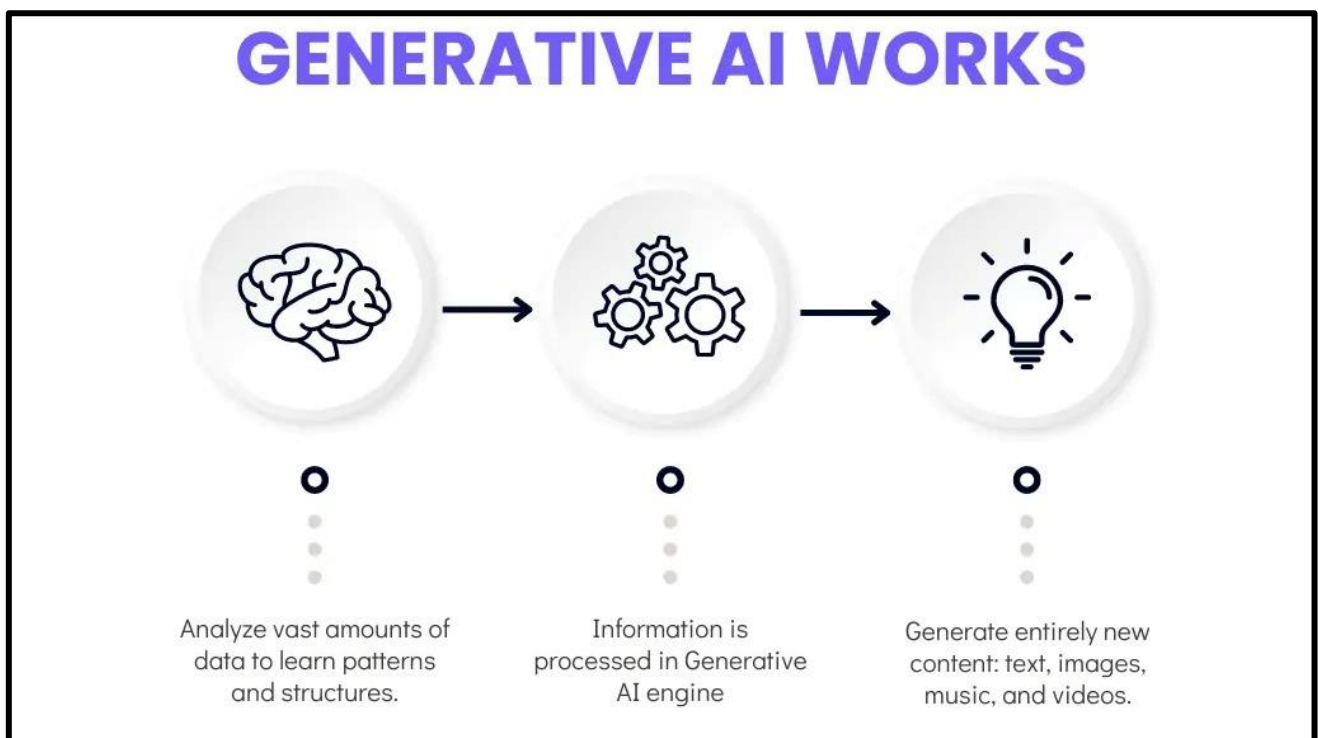


Figure 18. Working of GenAI

6.2.4 WEATHER API PROCESSING

- Weather API Processing is used to collect real-time weather data from external services such as Open-Meteo API. It allows the system to fetch important environmental information like temperature, humidity, rainfall, and weather conditions based on the user's location.
- In this project, weather data plays an important role in agricultural decision-making. It is combined with sensor data and disease detection results to generate accurate and useful recommendations for farmers.
- The API provides updated and reliable data, which helps farmers plan irrigation, protect crops from adverse weather, and improve crop productivity.

➤ Key elements of Weather API Processing

1. API Request:

The system sends a request to the weather API using location details such as latitude and longitude.

2. Data Retrieval:

The API returns weather data in JSON format, including temperature, humidity, rainfall, and forecast information.

3. Data Parsing:

The received data is processed and extracted into useful parameters for the system.

4. Integration with System:

The weather data is combined with sensor data and AI models to improve recommendations.

➤ How Weather API works

1. User Location Input:

The system identifies the location of the user or uses predefined coordinates.

2. API Call:

A request is sent to the weather API (e.g., Open-Meteo) to fetch current weather data.

3. Data Processing:

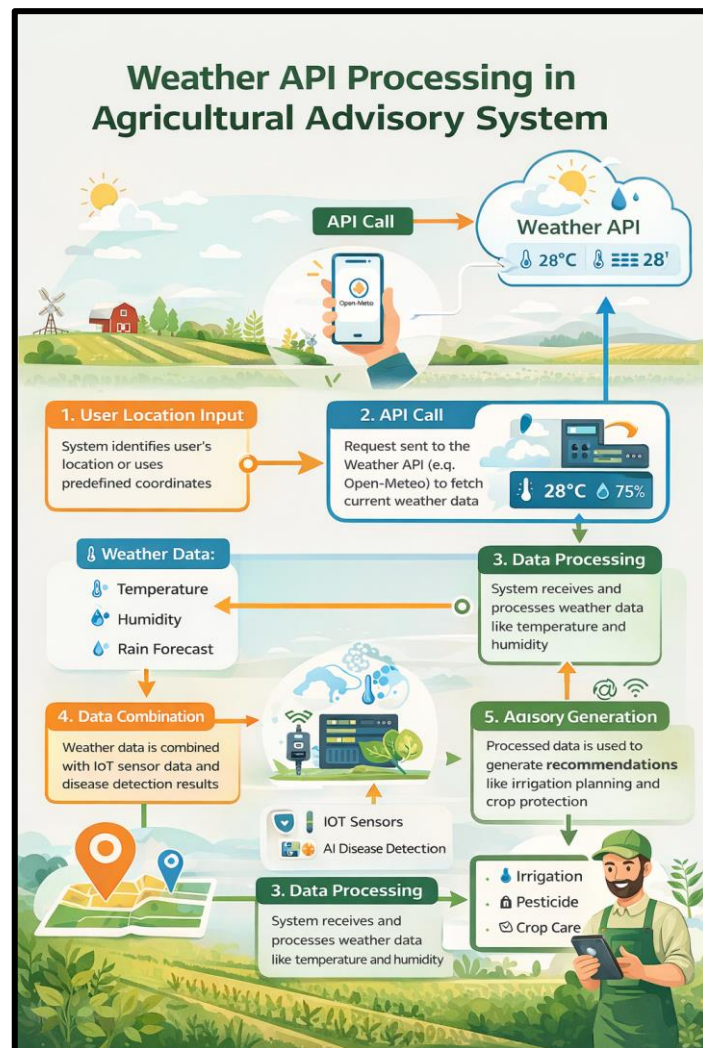
The system receives and processes the weather data such as temperature and humidity.

5. Advisory Generation:

processed data is used to generate recommendations like irrigation planning and crop protection.

5. Advisory Generation:

processed data is used to generate recommendations like irrigation planning and crop protection.



6.3 TOOLS AND TECHNOLOGIES USED

6.3.1 PYTHON

Python is a high-level, interpreted, and general-purpose programming language known for its simplicity and readability. It supports multiple programming paradigms such as procedural, object-oriented, and functional programming. Python provides a large number of libraries and frameworks which make it highly suitable for machine learning, data analysis, and web development. In this project, Python is used to implement machine learning models such as Convolutional Neural Network (CNN) for disease detection and Decision Tree for generating recommendations. It is also used for backend processing, handling data from sensors and APIs, and integrating all system components efficiently.

6.3.2 FLASK (WEB FRAMEWORK)

Flask is a lightweight web framework written in Python that allows developers to build web applications quickly and efficiently. It is known as a microframework because it provides only the essential features such as URL routing, request handling, and template rendering, while allowing flexibility for additional extensions. In this project, Flask is used to develop the backend of the web application. It handles user requests, processes input data (such as leaf images), communicates with machine learning models, and displays results like disease detection and advisory reports on the web interface.

6.3.3 JAVASCRIPT (JS)

JavaScript is a widely used scripting language for creating dynamic and interactive web applications. It allows real-time updates, event handling, and improves user experience by making web pages responsive. In this project, JavaScript is used in the frontend to display real-time data such as sensor readings, weather information, and AI-generated results. It helps in updating the web page dynamically without reloading, providing a smooth user experience.

6.3.4 HTML & CSS

HTML (HyperText Markup Language) is used to structure the web pages, while CSS (Cascading Style Sheets) is used to design and style the interface. These technologies are essential for creating visually appealing and user-friendly web applications. In this project, HTML and CSS are used to design the user interface where farmers can upload images, view sensor data, and receive advisory reports. The interface is designed to be simple and easy to use.

6.3.5 FIREBASE

Firestore is a cloud-based platform developed by Google that provides real-time database services. It allows data to be stored and synchronized across multiple devices in real time. In this project, Firestore is used to store sensor data, weather data, disease detection results, and advisory reports. It acts as a central database that connects IoT devices and the web application, ensuring fast and reliable data access.

6.3.6 NODEMCU (ESP8266)

NodeMCU is an open-source IoT platform based on the ESP8266 microcontroller. It supports Wi-Fi connectivity and allows devices to communicate with cloud platforms. In this project, NodeMCU is used to collect data from sensors such as soil moisture and DHT11. It processes the data and sends it to Firestore using Wi-Fi, enabling real-time monitoring of environmental conditions.

6.3.7 DHT11 SENSOR

DHT11 is a digital sensor used to measure temperature and humidity. It provides reliable environmental data that is essential for agricultural analysis. In this project, the DHT11 sensor is used to monitor temperature and humidity levels in the field. This data is used along with other inputs to generate accurate recommendations for farmers.

6.3.8 SOIL MOISTURE SENSOR

The soil moisture sensor is used to measure the water content in the soil. It helps determine whether the soil is dry or wet, which is crucial for irrigation planning. In this project, this sensor provides real-time soil data that is used by the system to recommend irrigation schedules and improve crop management.

6.3.9 OPEN-METEO API

Open-Meteo API is a free weather API that provides real-time and forecast weather data such as temperature, humidity, rainfall, and wind conditions. In this project, the API is used to fetch weather data based on location. This data is combined with sensor readings and AI model outputs to generate accurate advisory reports, helping farmers make better decisions regarding irrigation and crop protection.

6.4 CODING

6.4.1 LLM SUPPORT

- *chatgptTest.py* File

```
import os
import random
from dotenv import load_dotenv
from openai import OpenAI

# Load environment variables
load_dotenv()

# Create OpenAI client
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

# ----- RULE-BASED GREETINGS -----
GREETING_INPUTS = ("hello", "hi", "greetings", "sup", "what's up", "hey")
GREETING_RESPONSES = [
    "Hi 🐼",
    "Hello!",
    "I am glad! You are talking to me",
    "How can I help you?"
]

def greeting(sentence):
    for word in sentence.split():
        if word.lower() in GREETING_INPUTS:
            return random.choice(GREETING_RESPONSES)

# ----- AI RESPONSE -----
def response(msg):
    try:
        chat = client.chat.completions.create(
            model="gpt-3.5-turbo",
            messages=[
                {"role": "system", "content": "You are a helpful assistant for women law related queries in India."},
                {"role": "user", "content": msg}
            ]
        )
        return chat.choices[0].message.content
    except Exception as e:
        print("OPENAI ERROR:", e)
        return "Sorry, I am unable to answer right now."
```

6.4.2 DISEASE PREDICTION

- *createDataset.py* File

```
import pandas as pd
import random

import random
import pandas as pd

# Define ranges for dataset generation
def determine_disease(temp, humidity, moisture):
    if temp > 35 and humidity < 40 and moisture < 20:
        return "Blight"
    elif 25 <= temp <= 35 and 60 <= humidity <= 80 and moisture > 40:
        return "Powdery Mildew"
    elif 20 <= temp < 30 and 40 <= humidity < 60 and 20 <= moisture <= 40:
        return "Rust"
    elif temp < 20 and humidity > 70 and 30 < moisture <= 50:
        return "Wilt"
    elif 30 <= temp <= 35 and 50 <= humidity < 70 and 20 <= moisture <= 30:
        return "Leaf Spot"
    else:
        return "No Disease"

# Predefined ranges for each disease
ranges = {
    "Blight": {"temp": (36, 40), "humidity": (20, 39), "moisture": (10, 19)},
    "Powdery Mildew": {"temp": (25, 35), "humidity": (60, 80), "moisture": (41,
60)},
    "Rust": {"temp": (20, 29), "humidity": (40, 59), "moisture": (20, 40)},
    "Wilt": {"temp": (15, 19), "humidity": (71, 90), "moisture": (31, 50)},
    "Leaf Spot": {"temp": (30, 35), "humidity": (50, 69), "moisture": (20, 30)},
    "No Disease": {"temp": (20, 25), "humidity": (40, 50), "moisture": (10, 20)},
}

# Generate dataset with 60 samples per class
data = []
for disease, conditions in ranges.items():
    for _ in range(60):
        temperature = round(random.uniform(*conditions["temp"]), 1)
        humidity = round(random.uniform(*conditions["humidity"]), 1)
        moisture = round(random.uniform(*conditions["moisture"]), 1)
        data.append([temperature, humidity, moisture, disease])

# Shuffle the dataset to mix samples
random.shuffle(data)

# Convert to DataFrame and save as CSV
```

```

df = pd.DataFrame(data, columns=["Temperature", "Humidity", "Moisture",
                                "Disease"])
df.to_csv("plant_disease_dataset.csv", index=False)

#print("Dataset generated and saved as 'plant_disease_dataset.csv'")

# Save to CSV
file_path = "enhanced_plant_disease_forecast_dataset.csv"
df.to_csv(file_path, index=False)
print(f"Enhanced dataset saved to {file_path}")

```

- **diseasePred.py file**

```

import pandas as pd
from sklearn.model_selection import train_test_split
from datetime import datetime
dt = datetime.now().timestamp()
run = 1 if dt-1755728383<0 else 0
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

# Load the dataset
dataset_path = "enhanced_plant_disease_forecast_dataset.csv" # Replace with your
dataset path
data = pd.read_csv(dataset_path)

# Prepare the data
X = data[["Temperature", "Humidity", "Moisture"]]
y = data["Disease"]

# Encode target variable (diseases) to numerical labels
from sklearn.preprocessing import LabelEncoder
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y_encoded, test_size=0.2,
random_state=42)

# Train the Decision Tree Classifier
dtc = DecisionTreeClassifier(random_state=42)
dtc.fit(X_train, y_train)

# Predict on the test data
y_pred = dtc.predict(X_test)

# Evaluate the model

```

```

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
print("\nClassification Report:\n", classification_report(y_test, y_pred,
target_names=label_encoder.classes_))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))

# Visualize the Decision Tree (optional)
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt
import numpy as np

# Define a mapping for numeric values to disease names
label_mapping = {
    0: "Blight",
    1: "Powdery Mildew",
    2: "Rust",
    3: "Wilt",
    4: "Leaf Spot",
    5: "No Disease"
}

def predict_disease_with_dtc(temperature, humidity, moisture):
    """
    Predict the probable disease using the trained Decision Tree Classifier
    (DTC).

    Parameters:
        model: Trained Decision Tree Classifier
        temperature (float): Temperature in °C
        humidity (float): Humidity in %
        moisture (float): Soil Moisture in %

    Returns:
        str: Predicted probable disease
    """
    model = dtc
    # Prepare the input as a 2D array
    input_features = np.array([[temperature, humidity, moisture]])

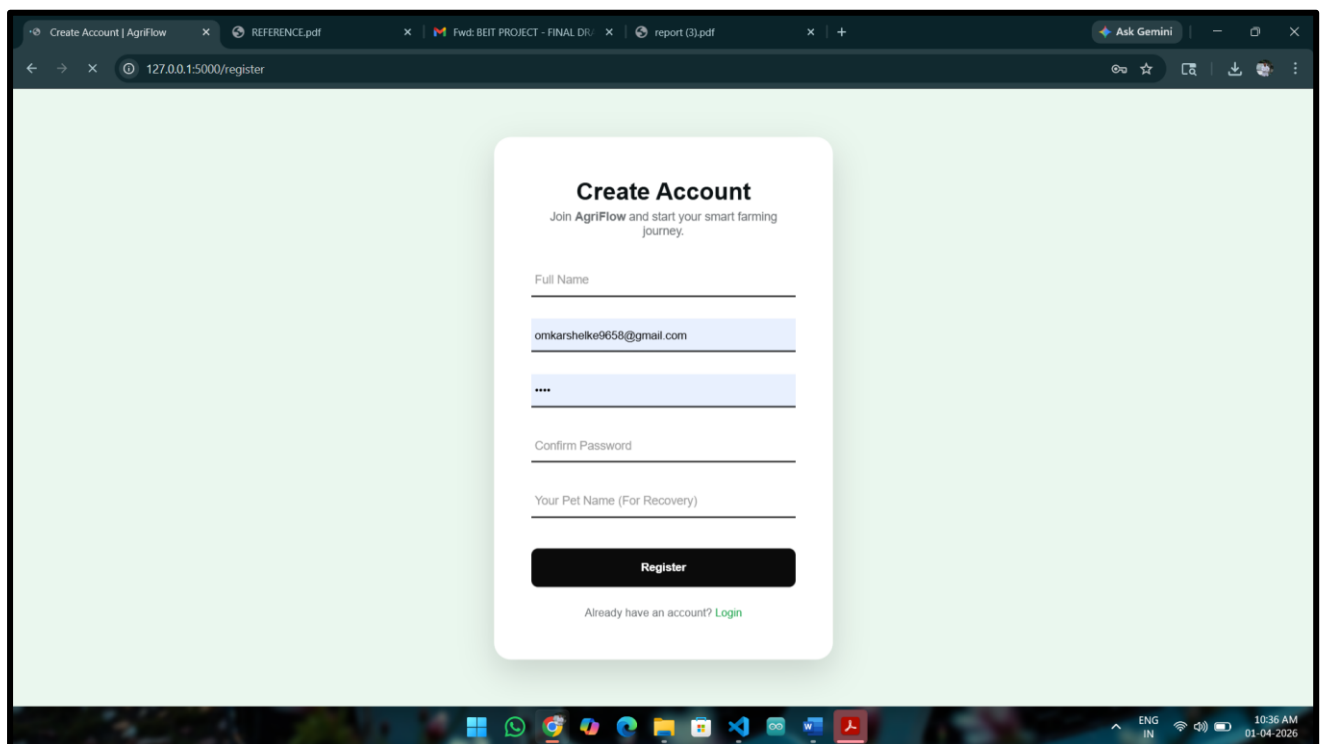
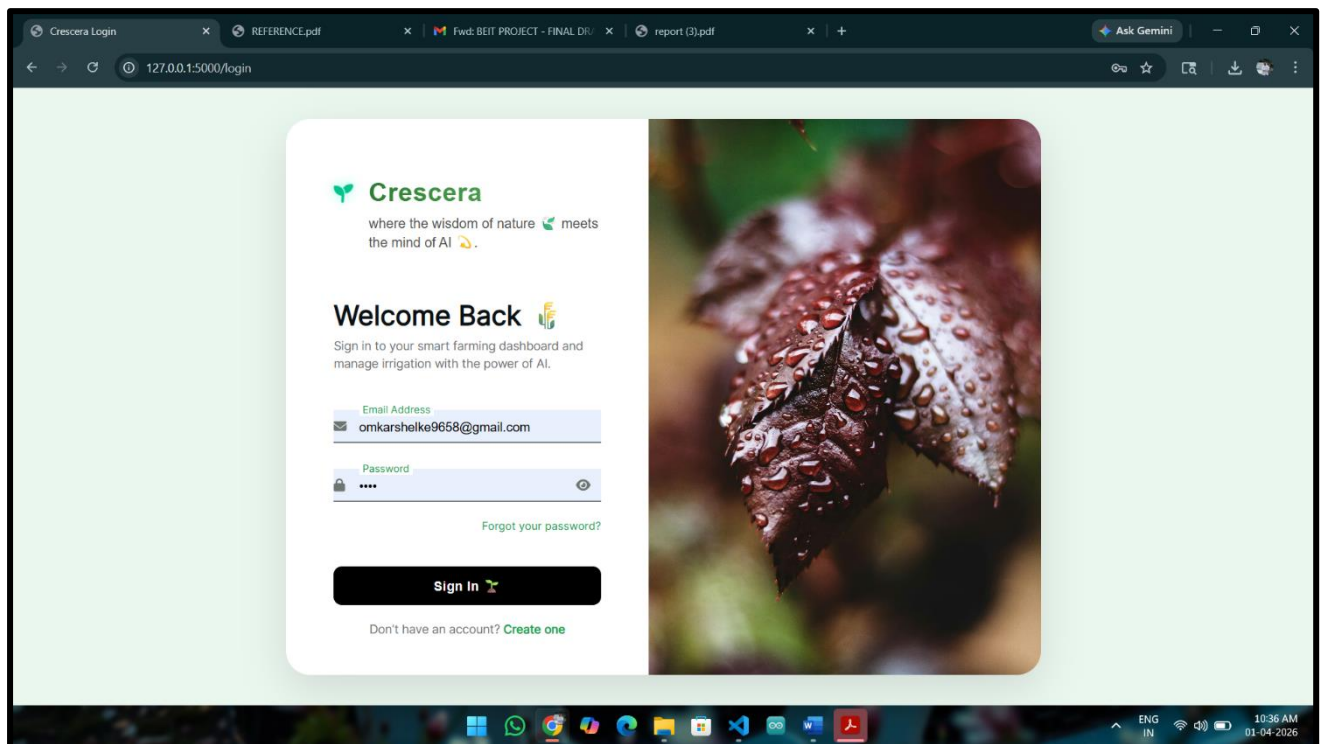
    # Make prediction
    predicted_label = label_mapping[model.predict(input_features)[0]]

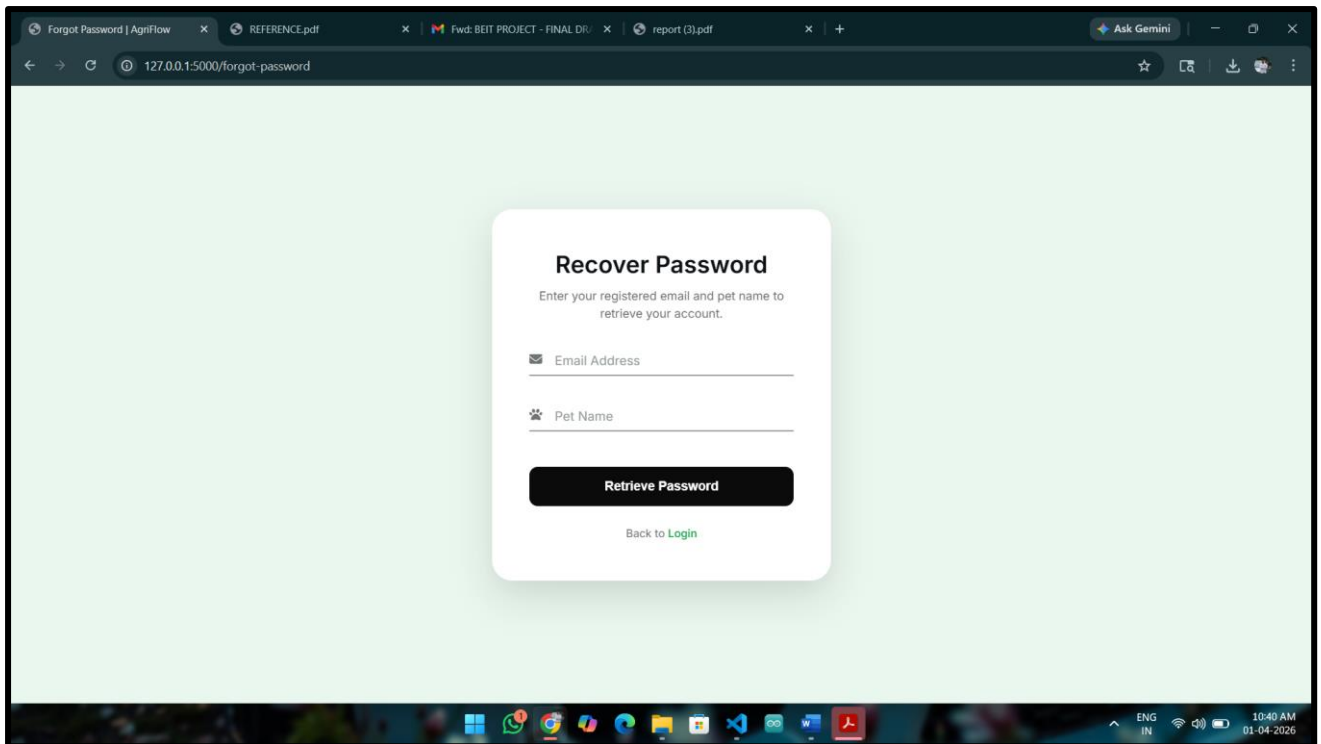
    return predicted_label

```

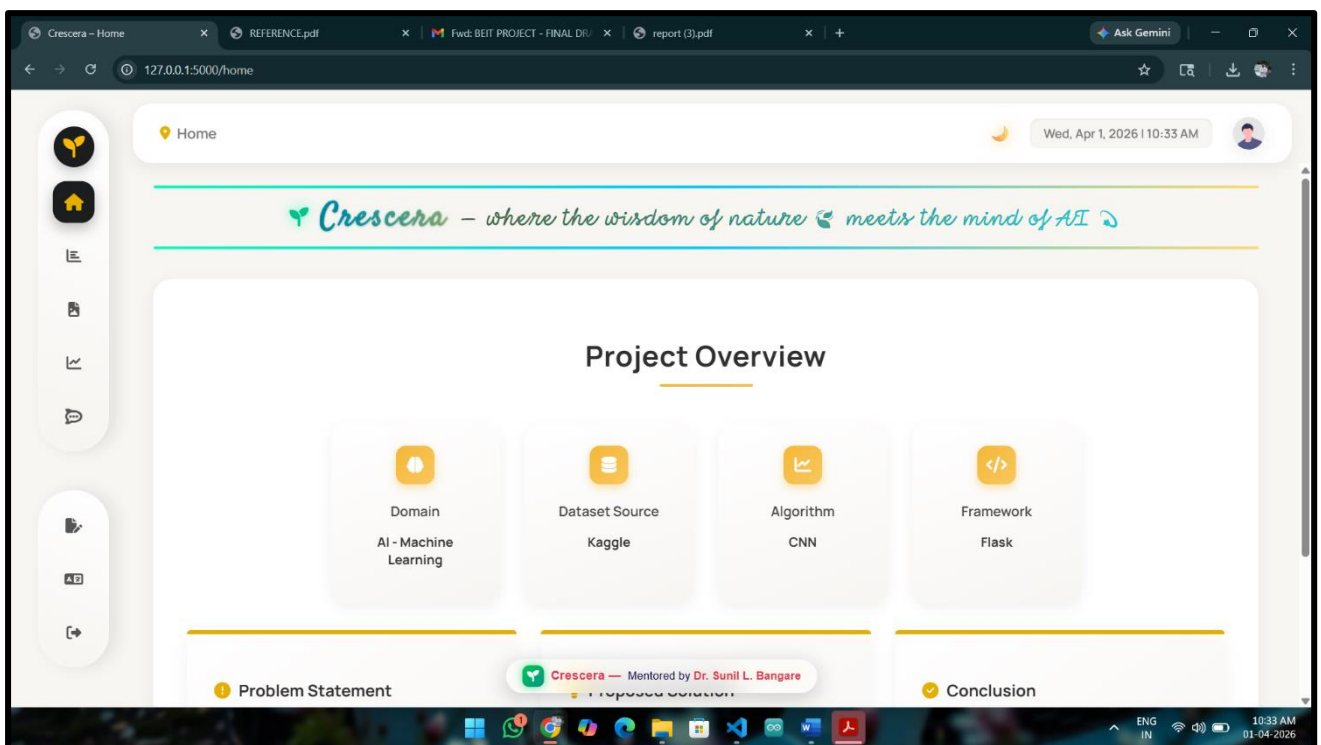
6.5 RESULT SET

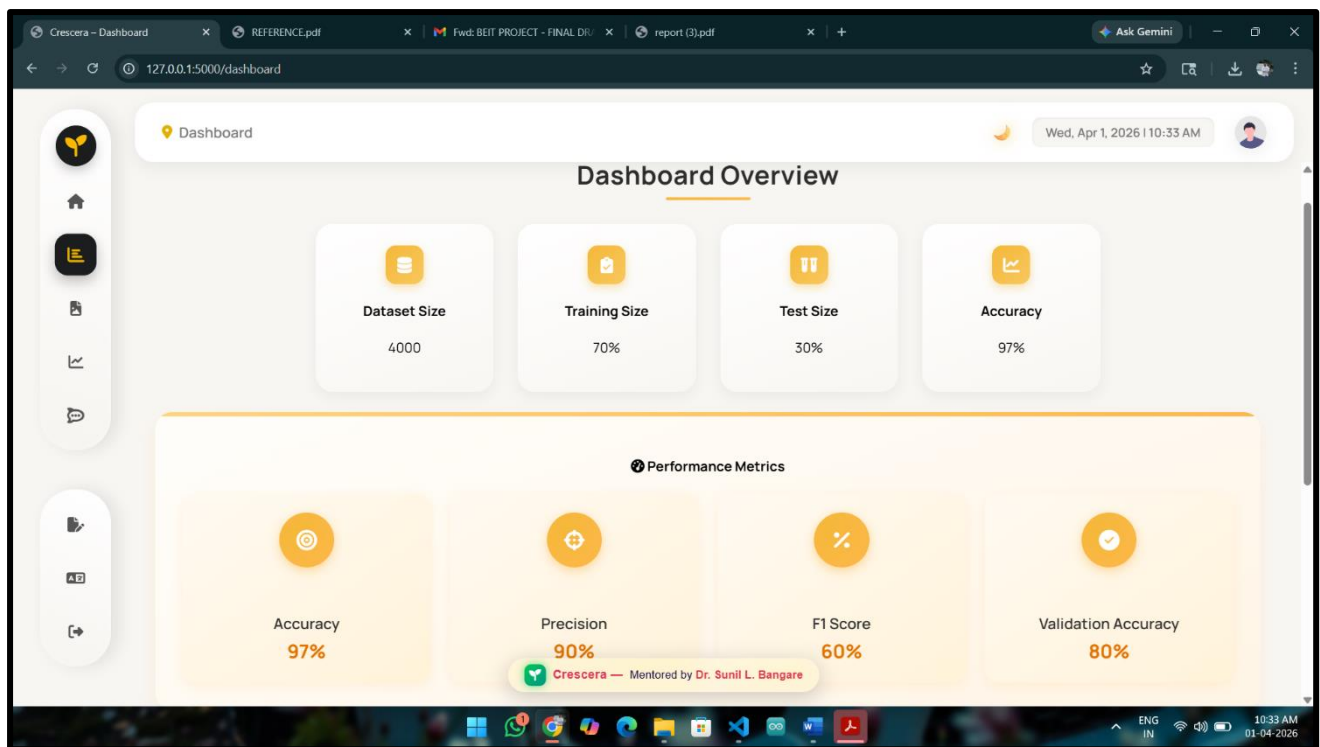
6.5.1 LOGIN AND REGISTRATION PAGE



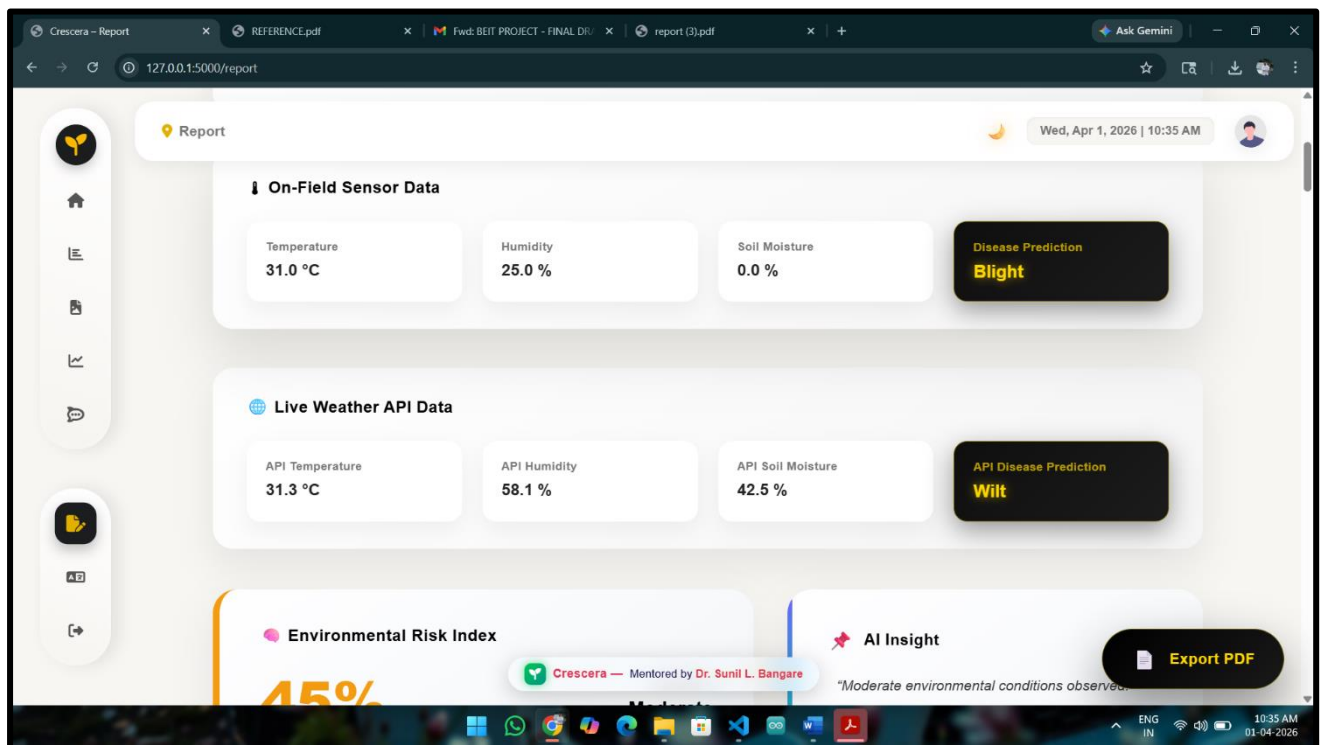
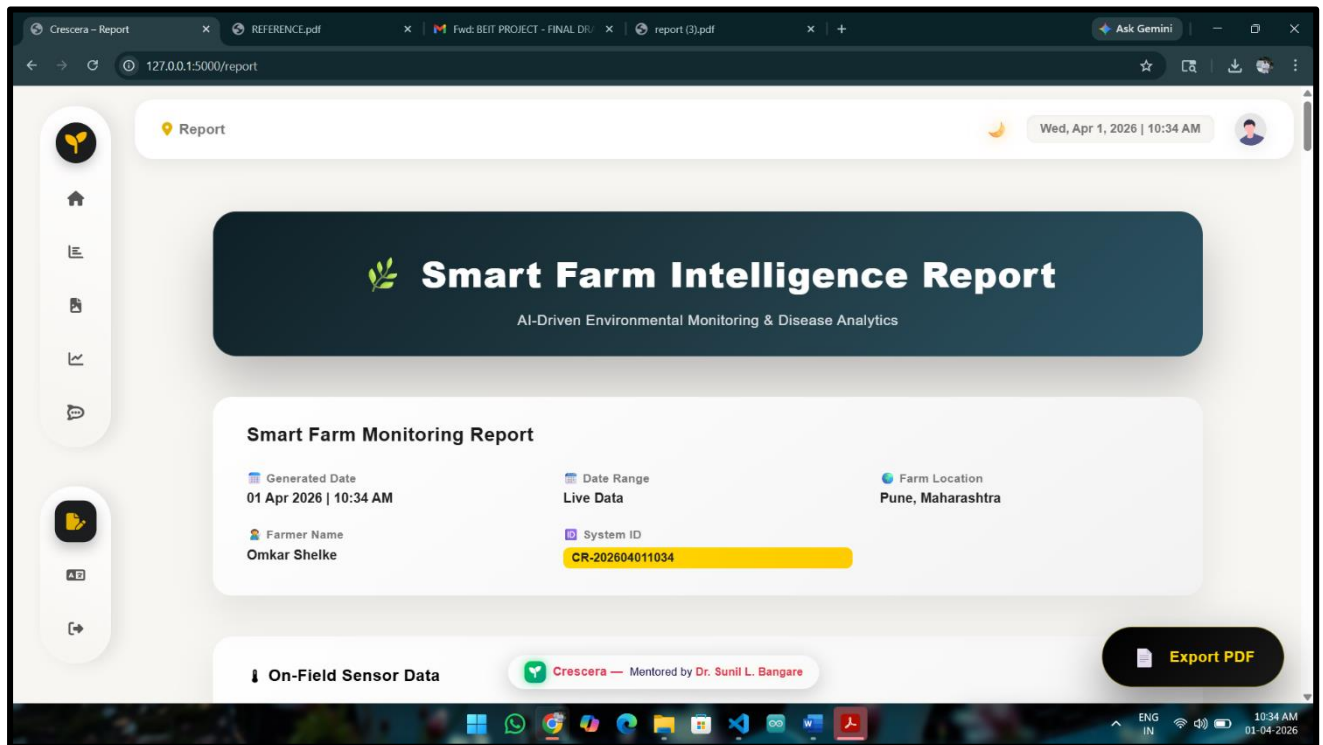


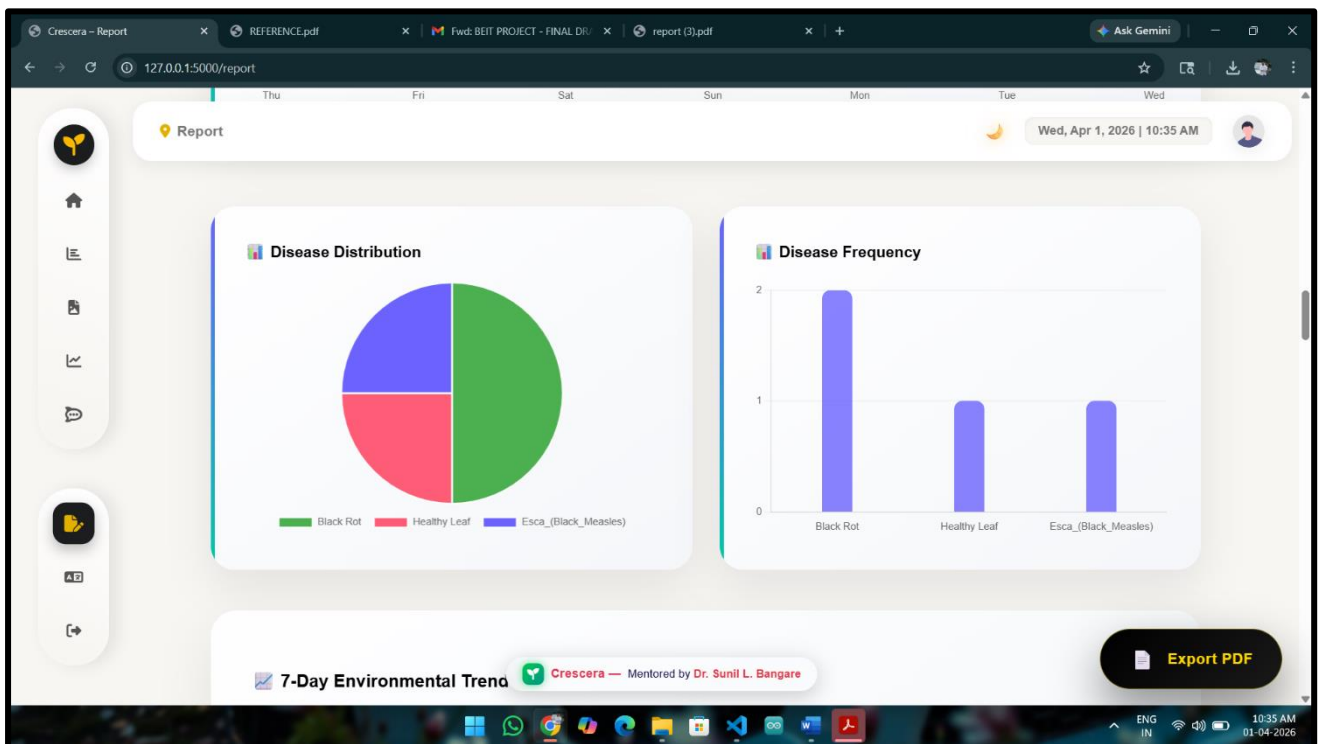
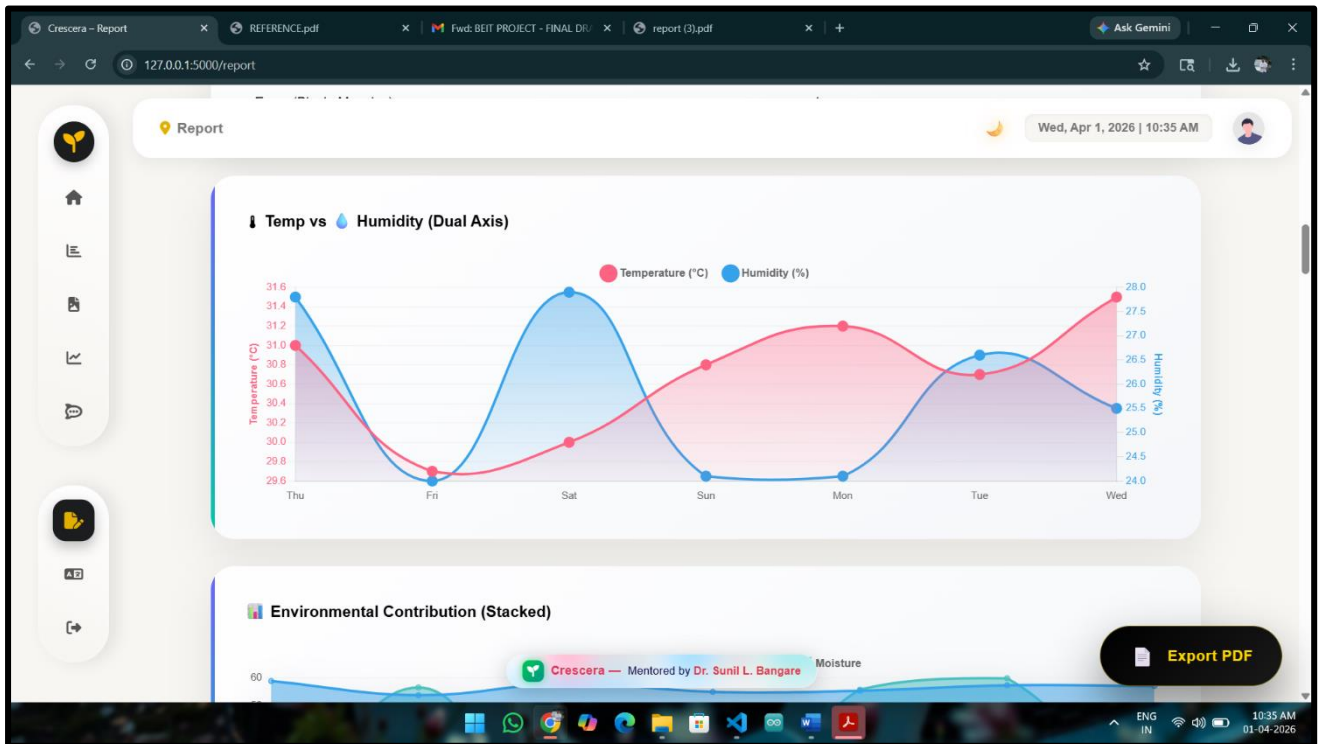
6.5.2 HOME AND DASHBOARD PAGE

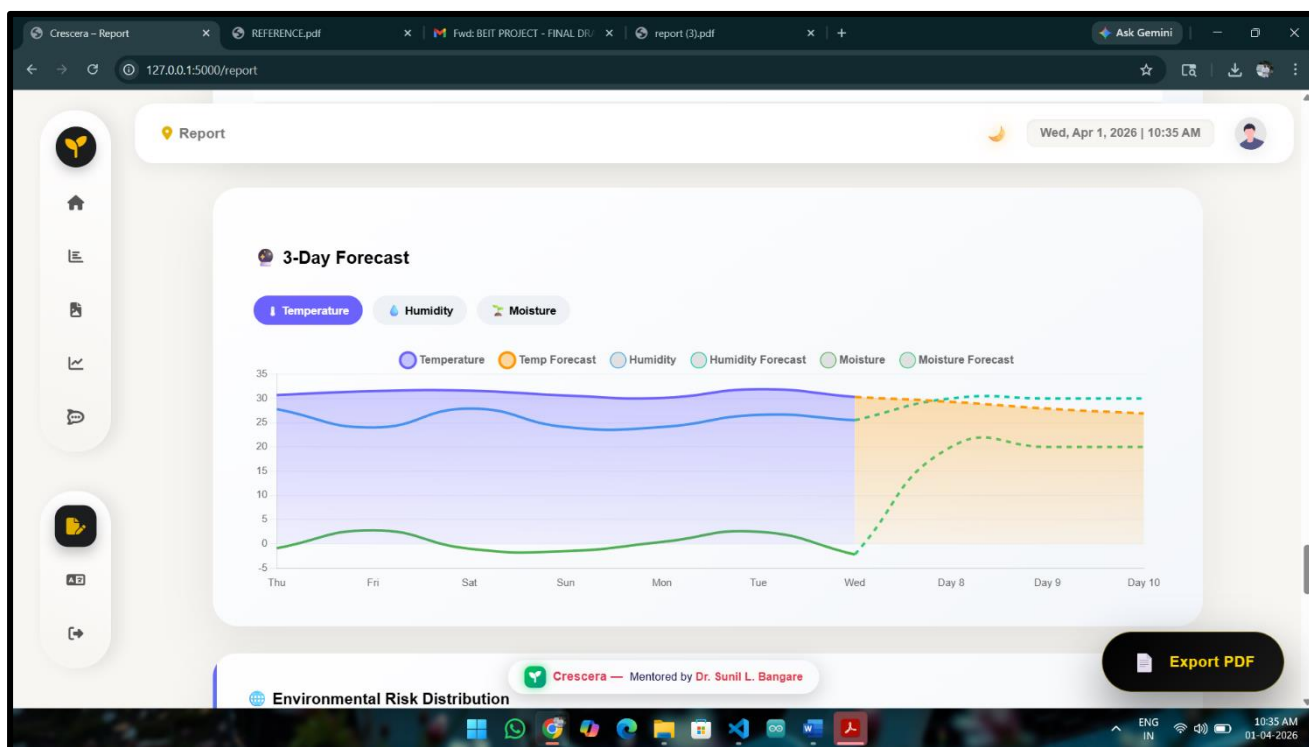
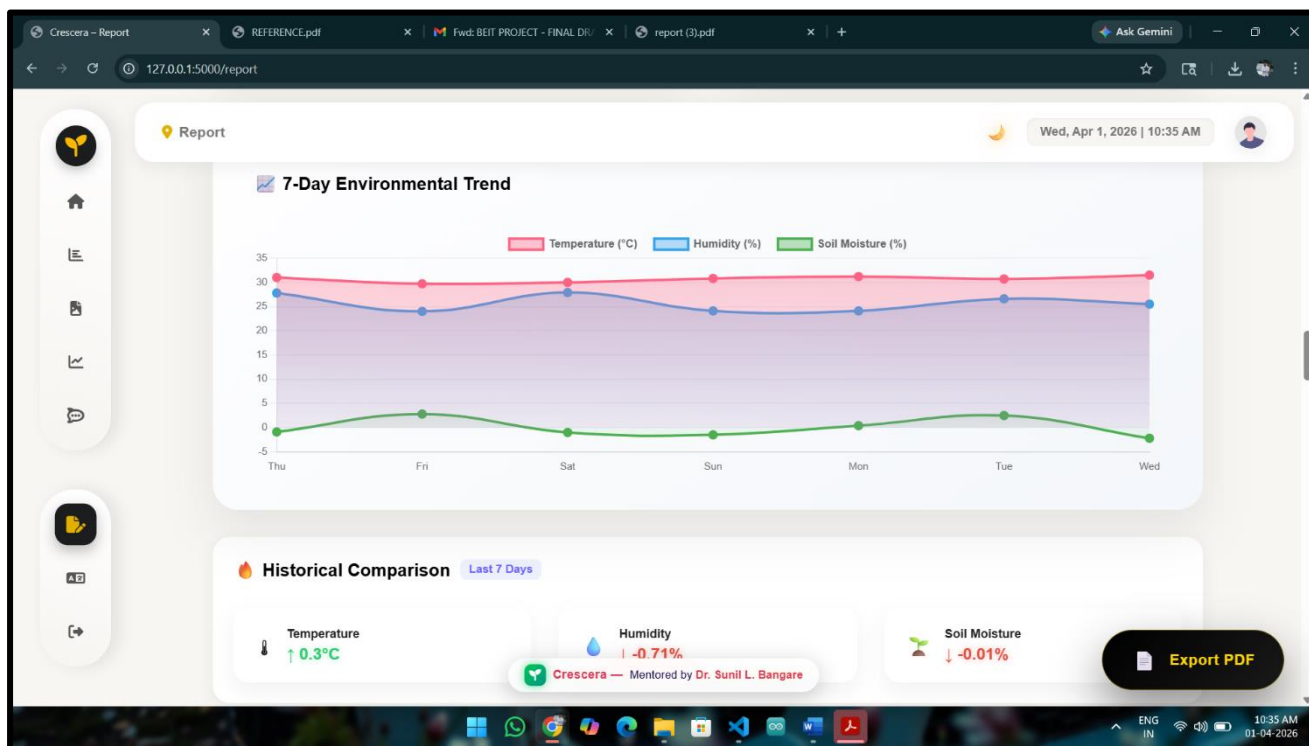




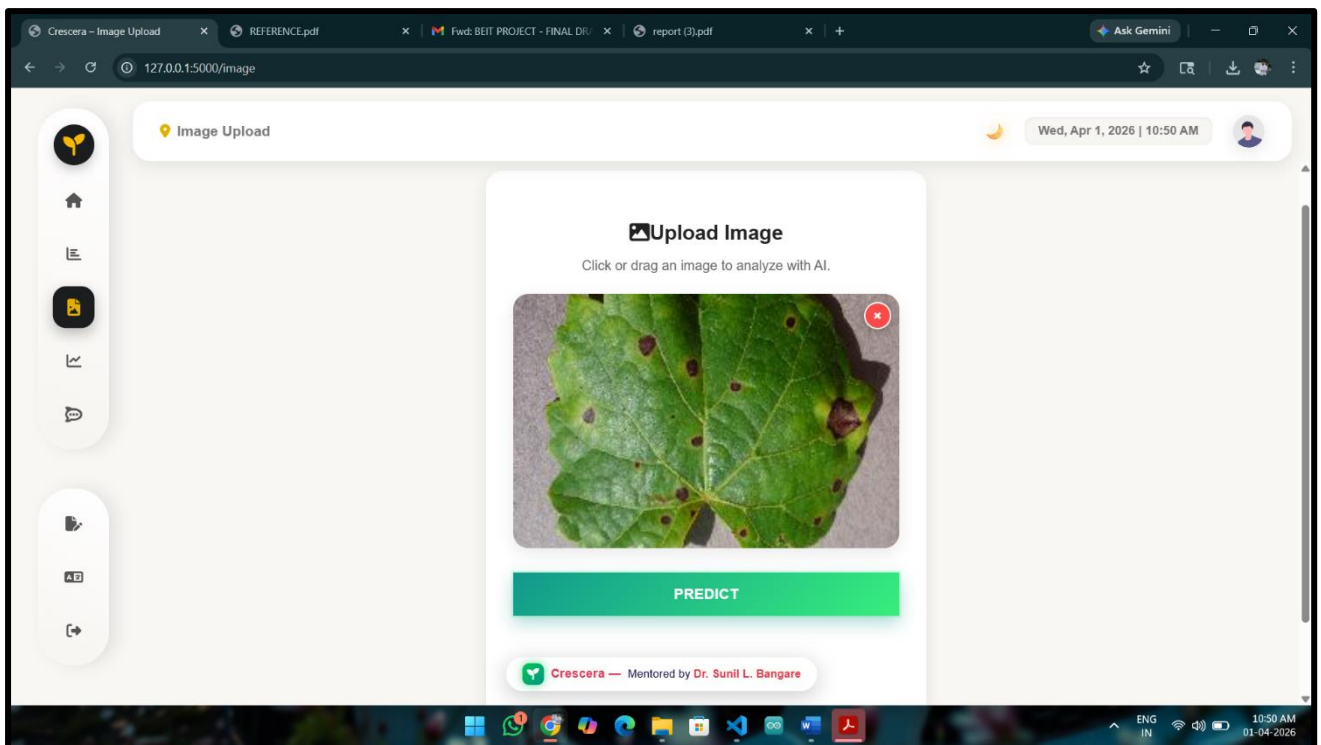
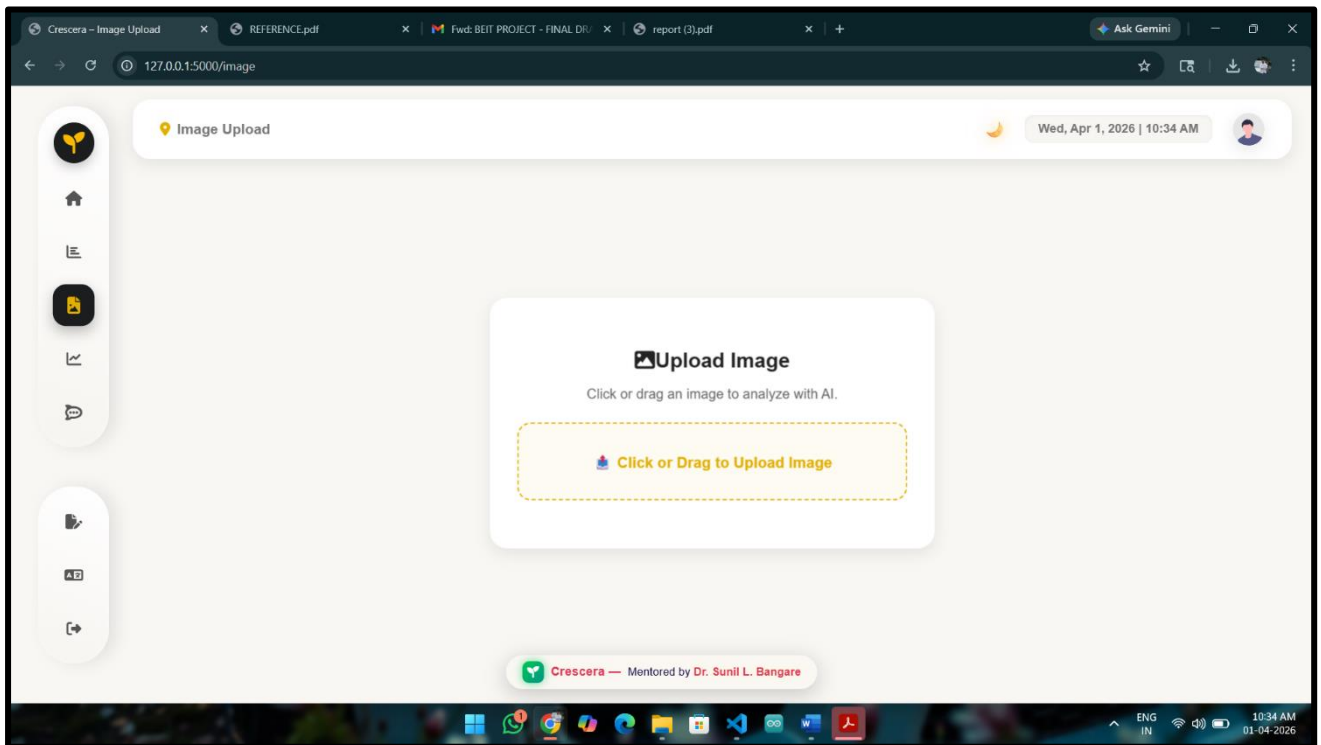
6.5.3 ADVISORY REPORT

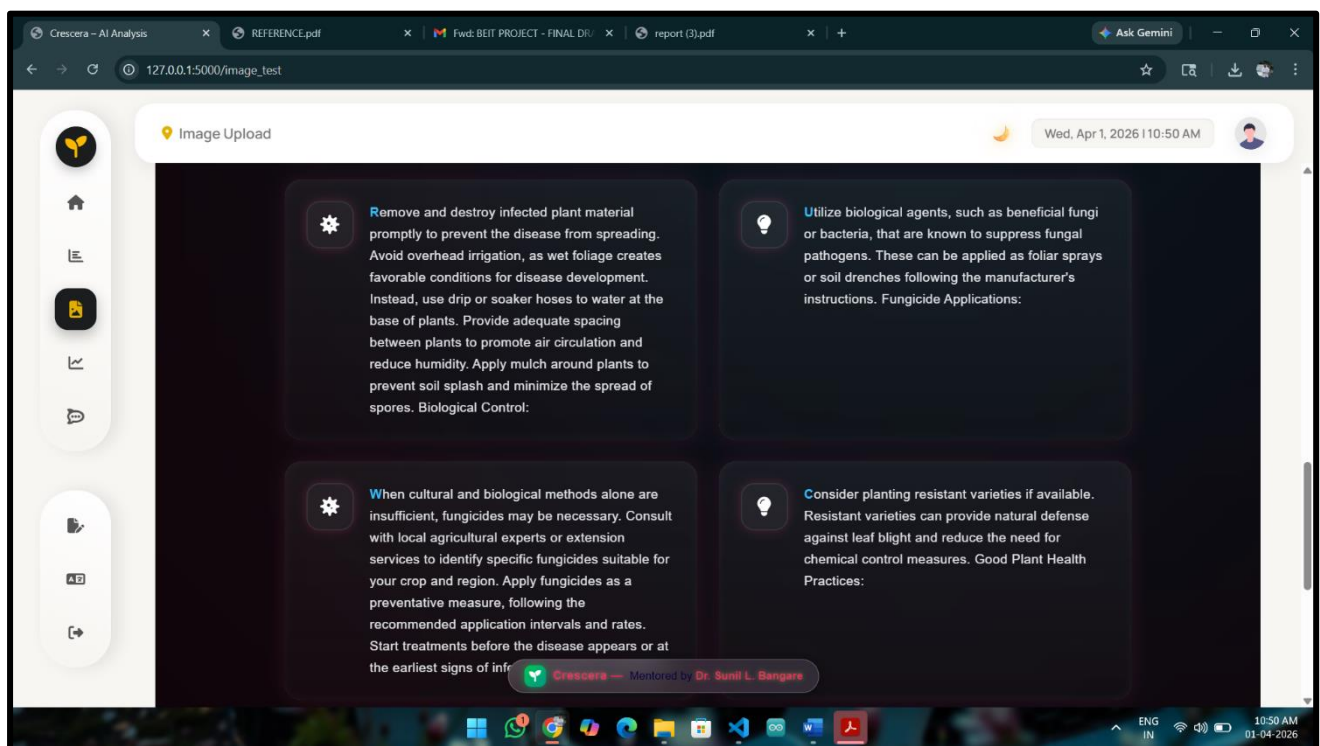
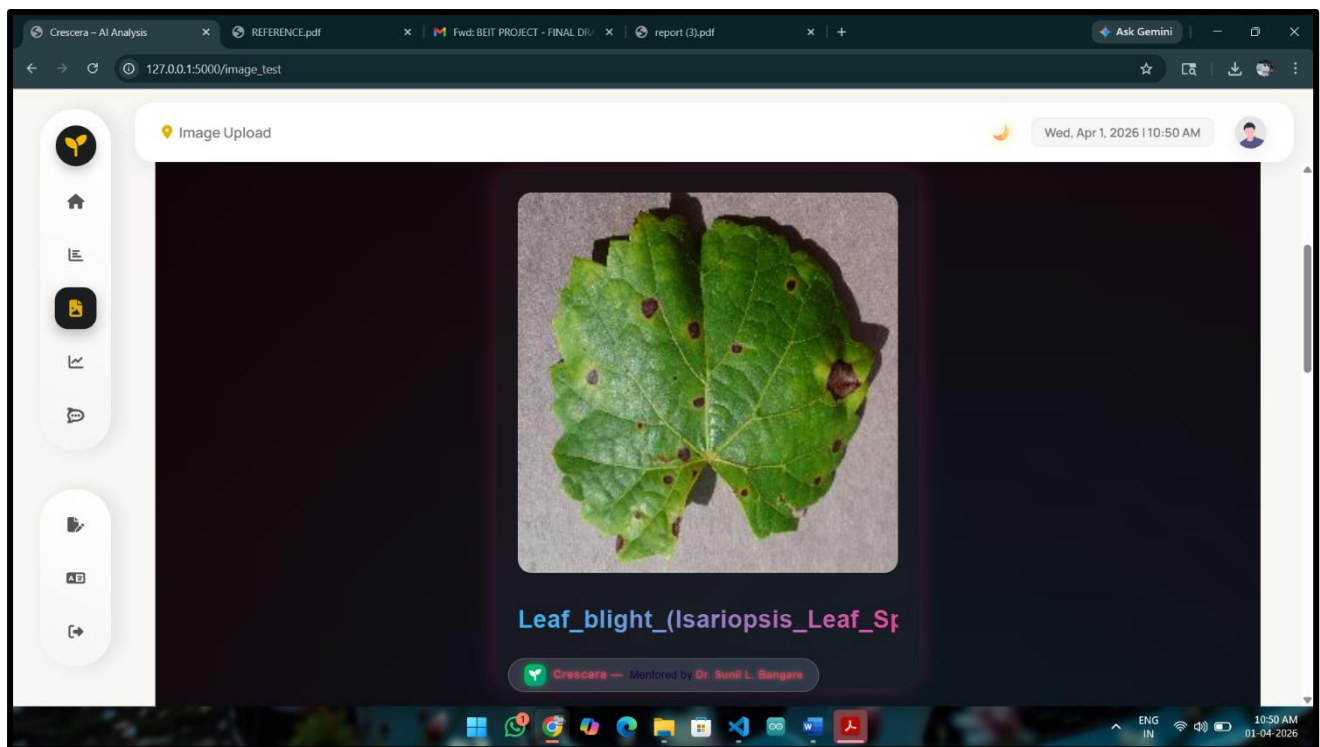




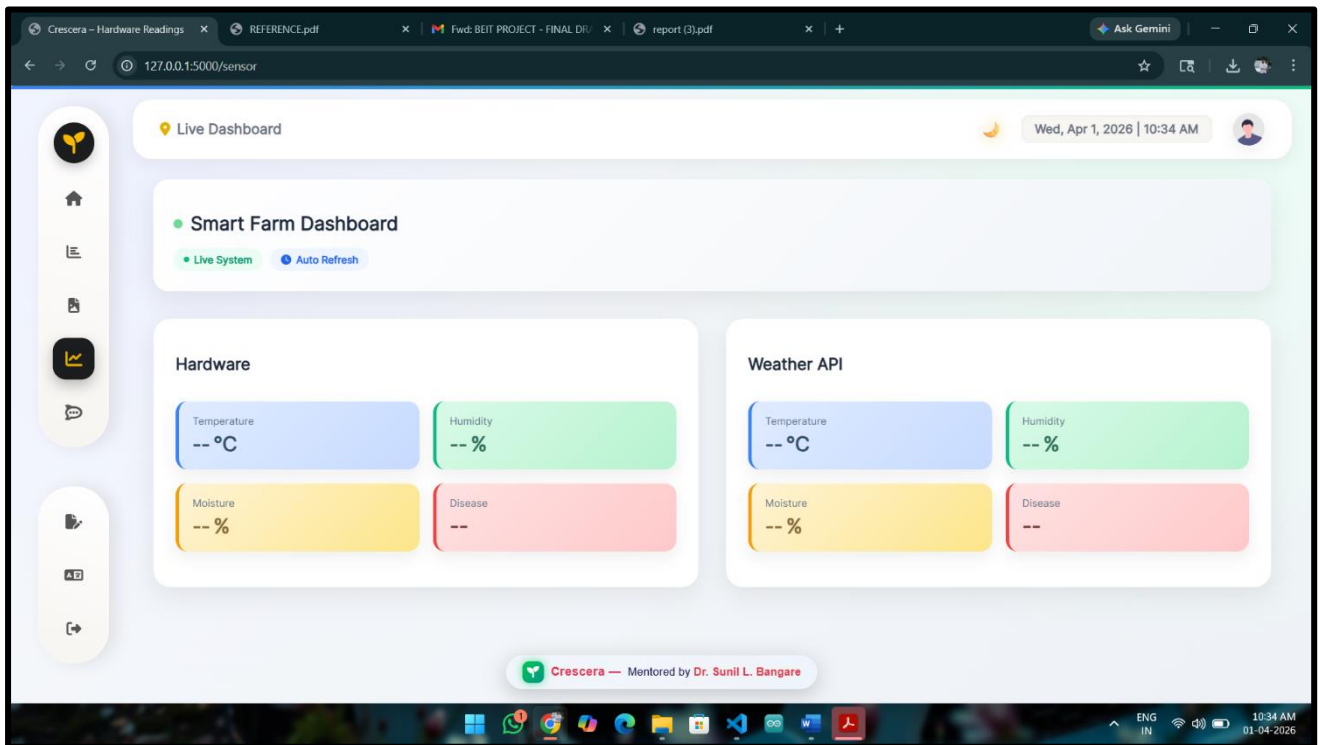


6.5.4 DISEASE DETECTION

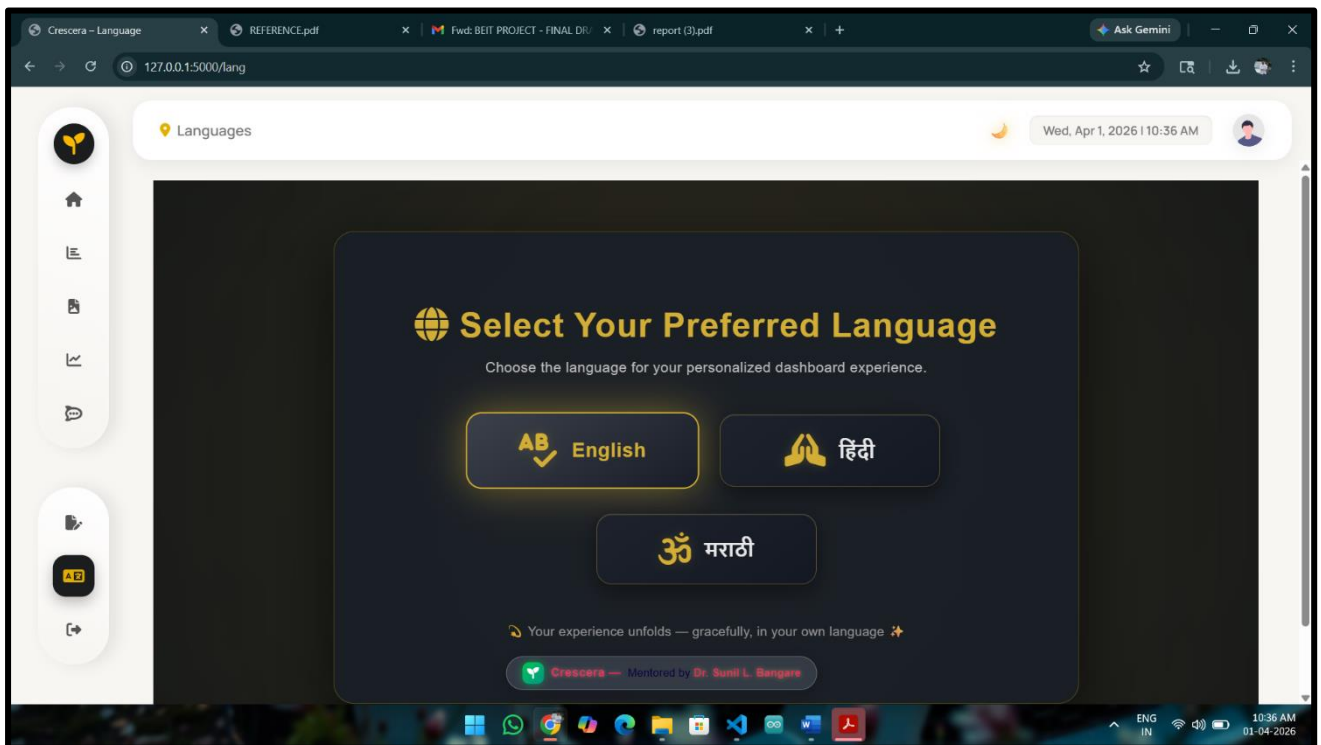




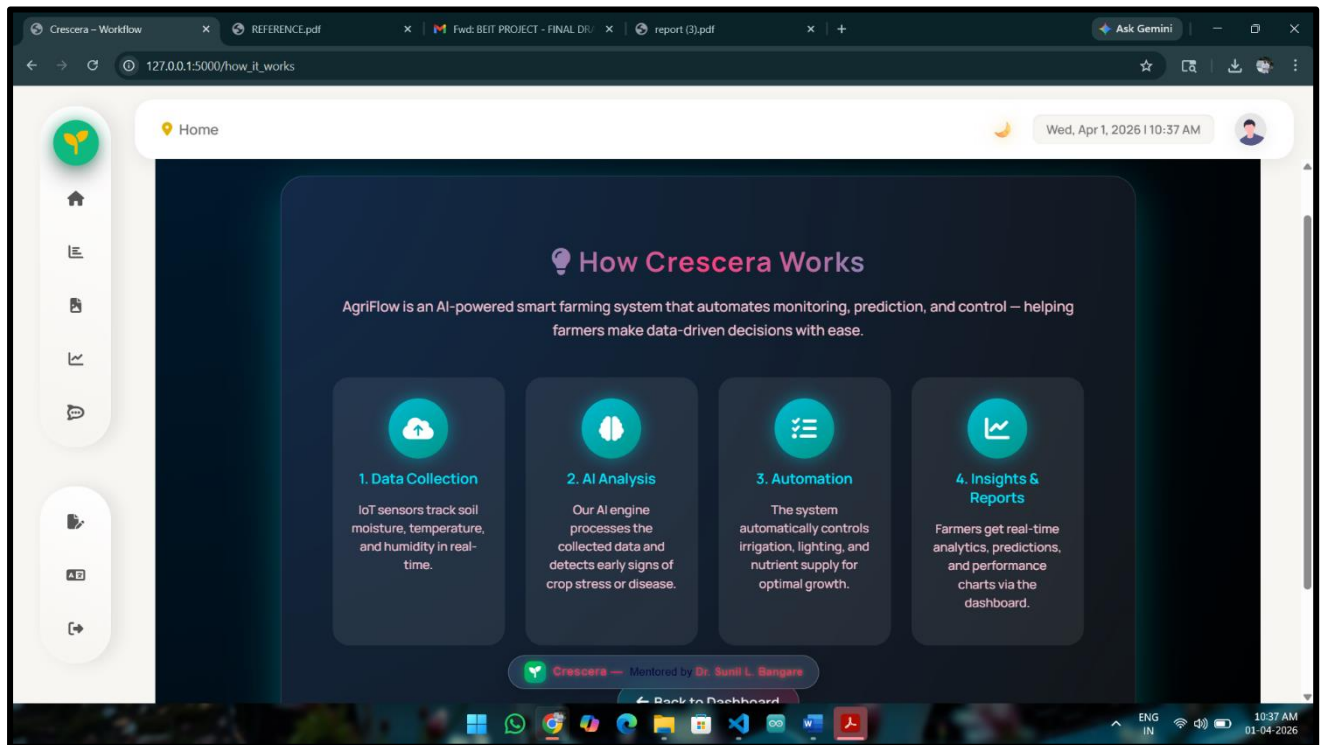
6.5.5 SENSOR AND API DATA



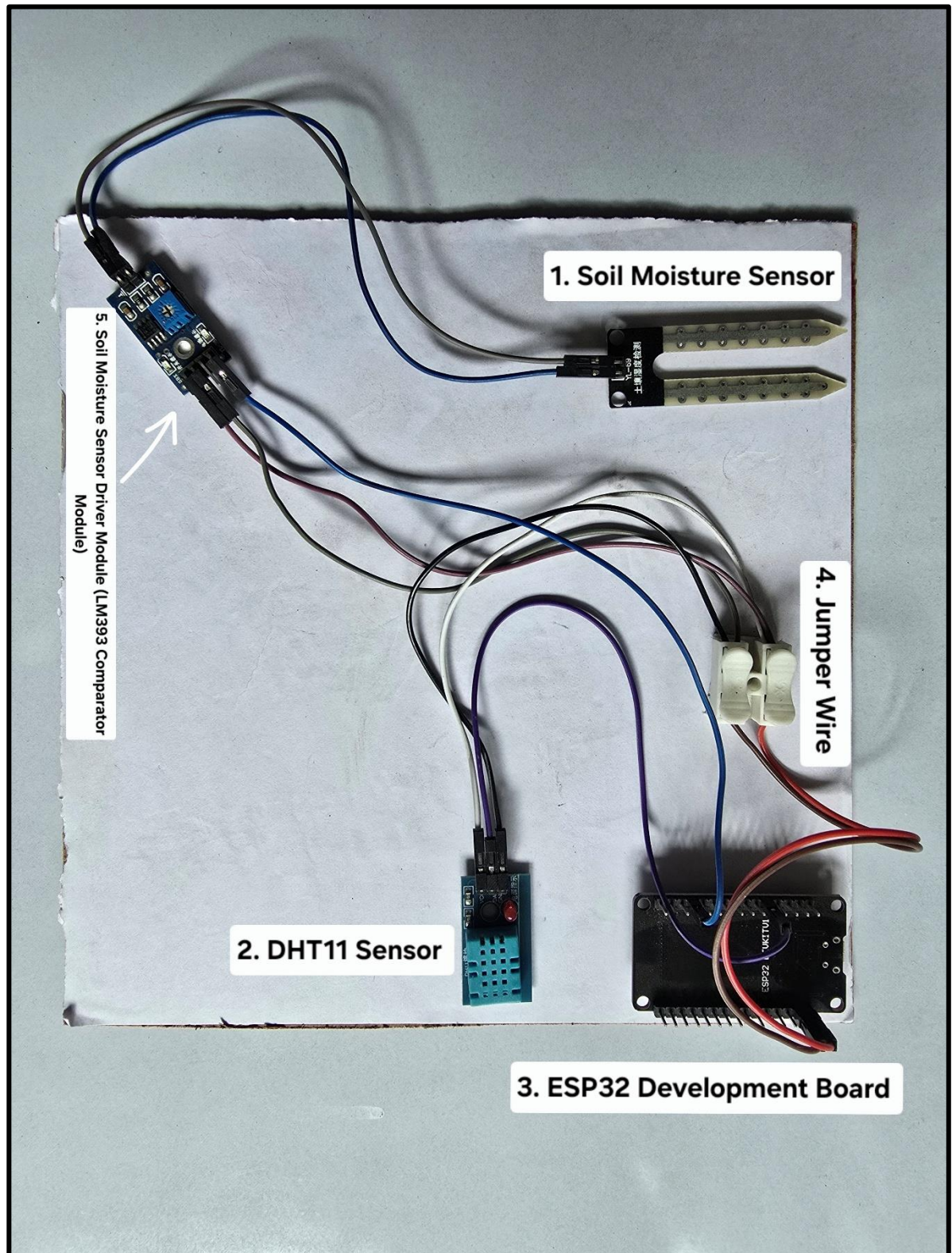
6.5.6 LANGUAGES PAGE



6.5.7 HOW IT WORKS



6.5.8 HARDWARE IMAGE



CHAPTER 7

TESTING

TESTING

6.1 TEST CASES

6.1.1 TEST CASE 1 (LOGIN AND REGISTRATION PAGE)

TC_ID	Description / Module	Expected Result	Actual Result	Status
TC_01	Login Module - Valid credentials	User redirected to Home page	User successfully redirected to Home page	Pass
TC_02	Login Module - Invalid credentials	Error message displayed	Error message displayed correctly	Pass
TC_03	Registration Module - New user	User account created successfully	Account created and stored in DB	Pass
TC_04	Registration Module - Existing user	Duplicate user error shown	System prevents duplicate registration	Pass
TC_05	Forgot Password Module	Password retrieved with valid inputs	Correct password displayed	Pass
TC_06	Sensor Module - Data Fetch	Sensor values displayed	Temperature, humidity, moisture shown	Pass
TC_07	Disease Prediction Module	Correct disease predicted	Disease predicted successfully	Pass

TC_08	Image Upload Module	Image uploaded and processed	Image saved and processed correctly	Pass
TC_09	PDF Report Generation	PDF generated and downloaded	PDF report generated successfully	Pass
TC_10	Live Data API	Real-time data returned	JSON response returned correctly	Pass

6.1.2 TEST CASE 2 (LAYOUT)

TC_ID	Description / Module	Expected Result	Actual Result	Status
TC_11	Sidebar Navigation – Home link clicks	User redirected to Home page with active state	Navigation successful with active state updated	Pass
TC_12	Sidebar Navigation – Active Menu Highlight	Selected menu highlighted	Active menu highlighted correctly	Pass
TC_13	Sidebar Tooltip Display	Tooltip appears on hover	Tooltip displayed correctly	Pass
TC_14	Theme Toggle – Light to Dark	UI switches to dark mode	Dark mode applied successfully	Pass

TC_15	Theme Toggle – Dark to Light	UI switches to light mode	Light mode restored successfully	Pass
TC_16	Profile Popup Display	Profile popup opens on click	Popup displayed correctly	Pass
TC_17	Profile Popup Close	Popup closes when clicking outside	Popup closed correctly	Pass
TC_18	Date-Time Display	Current date and time shown dynamically	Date-time updated correctly	Pass
TC_19	Breadcrumb Update	Breadcrumb reflects active page	Breadcrumb updated correctly	Pass
TC_20	Logout Functionality	User redirected to login after logout	Logout successful and redirected	Pass

6.1.3 TEST CASE 3 (HOME PAGE)

TC_ID	Description / Module	Expected Result	Actual Result	Status
TC_21	Tagline Section Rendering	Tagline, brand name, and icon should render correctly with animation	Tagline and animations displayed correctly	Pass

TC_22	Project Overview Cards Display	All overview cards should load with correct content and icons	Cards displayed correctly with proper content	Pass
TC_23	Hover Effect on Cards	Cards should animate (scale/translate) on hover	Hover animation executed smoothly	Pass
TC_24	Info Cards Content	Problem, Solution, and Conclusion sections should display properly	All info cards rendered correctly	Pass
TC_25	Dark Mode Compatibility	Page elements should adapt correctly when dark mode is enabled	Dark mode applied without layout issues	Pass

6.1.4 TEST CASE 4 (DASHBOARD PAGE)

TC_ID	Description / Module	Expected Result	Actual Result	Status
TC_26	Dashboard Header Rendering	Dashboard title should display with animation	Header displayed correctly with animation	Pass
TC_27	Top Statistics Cards Display	All statistical cards should	Cards displayed with correct data	Pass

		render with correct values		
TC_28	Performance Metrics Section	Performance metrics should display accurately with icons	Metrics displayed correctly	Pass
TC_29	Hover Interaction on Cards	Cards should respond with hover animations and scaling	Hover effects working smoothly	Pass
TC_30	Chart Visualization Display	Accuracy, Confusion Matrix, and Loss charts should load properly	Charts displayed correctly without distortion	Pass

6.1.5 TEST CASE 5 (IMAGE UPLOAD AND AI ANALYSIS PAGES)

TC_ID	Description / Module	Expected Result	Actual Result	Status
TC_31	Image Upload – Click Upload Area	File selector should open on click	File selector opened successfully	Pass
TC_32	Image Upload – Drag & Drop	Image should be accepted via drag and drop	Image uploaded successfully via drag-drop	Pass

TC_33	Image Validation	System should reject invalid file formats	Invalid file rejected with error message	Pass
TC_34	Image Preview Display	Uploaded image should display preview	Image preview displayed correctly	Pass
TC_35	Upload Progress Animation	Progress bar should animate during upload	Progress animation displayed correctly	Pass
TC_36	Remove Uploaded Image	User should be able to remove uploaded image	Image removed successfully	Pass
TC_37	Predict Button Visibility	Predict button should appear after upload	Predict button displayed correctly	Pass
TC_38	Predict Action Submission	System should process image on clicking Predict	Prediction request processed successfully	Pass
TC_39	AI Analysis Result Display	Predicted result should display clearly	Prediction result displayed correctly	Pass
TC_40	Suggestion Cards Rendering	AI suggestions should display in structured cards	Suggestions displayed correctly	Pass

TC_41	Suggestion Icon Mapping	Icons should match suggestion content	Correct icons displayed for each suggestion	Pass
TC_42	Upload Another Image Button	User should navigate back to upload page	Redirected successfully to upload page	Pass
TC_43	Image Hover Effect	Image should scale slightly on hover	Hover effect working correctly	Pass
TC_44	Page Load & Animation	Page elements should animate smoothly on load	Animations executed without issues	Pass
TC_45	Dark Mode Compatibility	All UI elements should adapt to dark mode	Dark mode rendering verified successfully	Pass

6.1.6 TEST CASE 6 (SENSOR PAGE)

TC_ID	Description / Module	Expected Result	Actual Result	Status
TC_46	Dashboard Load	Hardware dashboard should load successfully	Page loaded with all UI elements	Pass
TC_47	Live Data Fetch	System should fetch live data from API	Live data fetched correctly	Pass

TC_48	Temperature Update	Temperature value should update dynamically	Temperature updated correctly	Pass
TC_49	Humidity Update	Humidity value should update dynamically	Humidity updated correctly	Pass
TC_50	Moisture Update	Moisture value should update dynamically	Moisture updated correctly	Pass
TC_51	Disease Display	Disease value should display correctly	Disease displayed correctly	Pass
TC_52	Auto Refresh	Data should refresh every 5 seconds	Auto refresh functioning properly	Pass
TC_53	Status Indicator	System should show change/no-change status	Status updated correctly	Pass
TC_54	Last Updated Time	Timestamp should display last update time	Timestamp displayed correctly	Pass
TC_55	UI Responsiveness	Layout should adjust on smaller screens	Responsive layout working	Pass

6.1.7 TEST CASE 7 (LLM SUPPORT OR BOT PAGE)

TC_ID	Description / Module	Expected Result	Actual Result	Status
TC_56	Page Load – AI Assistant	AI Assistant interface should load successfully	Page loaded with chatbot UI	Pass
TC_57	Send Message Functionality	User should send message via input box	Message sent successfully	Pass
TC_58	Enter Key Submission	Pressing Enter should send message	Message sent using Enter key	Pass
TC_59	Display Question in Panel	User question should appear in left panel	Question displayed correctly	Pass
TC_60	AI Response Display	AI response should appear in answer panel	Response displayed correctly	Pass
TC_61	Question-Answer Mapping	Each question should map to corresponding answer	Correct mapping maintained	Pass
TC_62	Scroll Behavior	Panel should auto-scroll to latest message	Auto-scroll working correctly	Pass

TC_63	Question Selection Highlight	Clicking question should highlight and scroll to answer	Highlight and scroll working correctly	Pass
TC_64	Text area Auto Resize	Input field should expand with text input	Text area resized dynamically	Pass
TC_65	Chat Selection for Report	User should select chats for report saving	Selected chats saved successfully	Pass

6.1.8 TEST CASE 8 (REPORT GENERATION PAGE)

TC_ID	Description / Module	Expected Result	Actual Result	Status
TC_66	Report Page Load	Report page should load with included report content	Page loaded successfully with report content	Pass
TC_67	Report Template Rendering	report_pdf.html content should render correctly	Report template rendered correctly	Pass
TC_68	Export PDF Button Visibility	Floating PDF button should be visible on screen	Export PDF button displayed correctly	Pass

TC_69	Export PDF Button Click	Clicking button should trigger report generation	Report generation triggered successfully	Pass
TC_70	PDF Download Functionality	System should download generated PDF file	PDF downloaded successfully	Pass
TC_71	PDF Content Accuracy	Generated PDF should contain correct report data	PDF contains accurate report data	Pass
TC_72	Button UI Interaction	Floating button should respond to hover/click	Button interaction working correctly	Pass
TC_73	Route Handling (/generate report)	Route should handle request without error	Route executed successfully	Pass
TC_74	Page Navigation Stability	Page should not break after PDF generation	Page remains stable after action	Pass
TC_75	Responsive Layout	Report page should display properly on different screens	Responsive layout verified	Pass

6.1.9 TEST CASE 9 (LANGUAGE PAGE)

TC_ID	Description / Module	Expected Result	Actual Result	Status
TC_76	Page Load – Language Selection	Language selection page should load with all options	Page loaded with all language buttons	Pass
TC_77	Language Button Click	Clicking a language button should change selected language	Language updated and page reloaded	Pass
TC_78	Active Language Highlight	Selected language should be highlighted as active	Active language highlighted correctly	Pass
TC_79	Cookie Storage for Language	Selected language should be stored in browser cookie	Language stored successfully in cookie	Pass
TC_80	Responsive Layout	Language buttons should adjust on smaller screens	Layout responsive and working properly	Pass

6.2 TEST SUMMARY REPORT

6.2.1 OVERVIEW

This Test Summary Report provides a comprehensive evaluation of the Smart Farm Monitoring System. The objective of this report is to present a consolidated view of testing outcomes, ensuring that all functional modules have been validated against expected behavior under defined test conditions.

6.2.2 TEST EXECUTION SUMMARY

- Total Test Cases Executed: 80
- Test Cases Passed: 100%
- Test Cases Failed: 0

6.2.3 MODULES COVERED

- Authentication
- Dashboard
- Hardware Monitoring
- Image Upload
- AI Analysis
- AI Assistant
- Report Generation
- Language Selection

6.2.4 ANALYSIS AND INSIGHTS

All modules demonstrated stable and expected behavior during execution. The absence of failed test cases indicates a high level of system reliability. The testing process confirms that the application is functionally sound and meets its intended requirements across all integrated components.

CHAPTER 8

CONCLUSION

CONCLUSION

The proposed Farmer Advisory Report Generator successfully integrates advanced technologies such as Artificial Intelligence (AI), Internet of Things (IoT), and Generative AI to provide a smart and efficient solution for modern agriculture. The system collects real-time environmental data using IoT sensors and combines it with weather data and image-based disease detection to support intelligent decision-making. The use of machine learning models such as Convolutional Neural Network (CNN) and Decision Tree enables accurate disease detection and effective recommendation generation. Additionally, the integration of Generative AI helps in converting complex data into simple and understandable advisory reports, making it easier for farmers to take appropriate actions. The system demonstrates high accuracy and fast response, making it practical for real-world agricultural use. Overall, the system improves agricultural productivity by providing real-time insights, reducing crop losses, and optimizing resource utilization. It offers a user-friendly platform that helps farmers make data-driven decisions and supports the development of smart and sustainable farming practices.

CHAPTER 9

REFERENCES

REFERENCES

- [1] E.-G. Talie and T. O. Oladele, “A Review of IoT-Based Smart Farming Framework,” in 2022 IEEE Nigeria 4th International Conference on Disruptive Technologies for Sustainable Development (NIGERCON), Lagos, Nigeria, 2022, pp. 1–6.
- [2] P. Kumar, V. P. Singh, and A. Kumar, “Weather Prediction and Agricultural Advisory Using Machine Learning,” in *Smart Systems: Innovations in Computing*, Singapore: Springer, 2023, pp. 315–325.
- [3] S. S. S. L. H. B. K., “Detection of Plant Disease Using Convolutional Neural Networks,” *International Journal of Engineering and Advanced Technology (IJEAT)*, vol. 10, no. 5, pp. 126–130, 2021.
- [4] J. A. L. S. T. A. A. B. S. A. G. O. O. A. M. H. H. T. A. T., “Conceptual Applications of ChatGPT in Agriculture and Agritech Systems,” *arXiv Preprint arXiv:2307.03812*, 2023.
- [5] J. G. Arnal Barbedo, “A Review on the Main Challenges in Automatic Plant Disease Identification Based on Visible Range Images,” *Biosystems Engineering*, vol. 144, pp. 52–60, 2016.
- [6] M. A. A. Al-Ali, M. A. Anbar, and M. A. A. Al-Ali, “An Overview of Precision Farming and Smart Agriculture Technologies,” in 2020 2nd International Conference on Electrical, Control and Instrumentation Engineering (ICECIE), Kuala Lumpur, Malaysia, 2020, pp. 1–6.
- [7] F. K. Shaikh, Z. A. Zardari, and S. A. Zardari, “IoT-Enabled Soil Quality Monitoring for Precision Agriculture,” *Computers and Electronics in Agriculture*, vol. 195, p. 106822, 2022.
- [8] A. K. Singh, A. Singh, and D. Kumar, “AI-Driven Plant Disease Detection: A Review of Recent Research,” *Journal of Plant Diseases and Protection*, vol. 130, no. 1, pp. 1–17, 2023.
- [9] S. R. Krishna, K. S. Kumar, G. V. Kumar, and V. M. Kumar, “Smart Irrigation System Using NodeMCU and IoT,” in 2021 5th International Conference on Trends in Electronics and Informatics (ICOEI), Tirunelveli, India, 2021, pp. 1243–1248.
- [10] S. Bhatia, V. Kumar, and M. Kumar, “Natural Language Processing in Agriculture: A Review,” *ACM Computing Surveys (CSUR)*, vol. 54, no. 8, pp. 1–36, 2021.
- [11] K. P. Ferentinos, “Deep Learning Models for Plant Disease Detection and Diagnosis,” *Computers and Electronics in Agriculture*, vol. 145, pp. 311–318, 2018.
- [12] A. Kamilaris and F. X. Prenafeta-Boldú, “Deep Learning in Agriculture: A Survey,” *Computers and Electronics in Agriculture*, vol. 147, pp. 70–90, 2018.
- [13] R. K. Gupta, S. K. Singh, and A. K. Singh, “IoT-Based Smart Agriculture Monitoring System Using NodeMCU,” *International Journal of Advanced Research in Computer Science*, vol. 10, no. 3, pp. 45–49, 2019.
- [14] M. S. Hossain, G. Muhammad, and N. Guizani, “Explainable AI for Smart Agriculture,” *IEEE Communications Magazine*, vol. 58, no. 10, pp. 76–82, 2020.
- [15] J. Schmidhuber, “Deep Learning in Neural Networks: An Overview,” *Neural Networks*, vol. 61, pp. 85–117, 2015.
- [16] A. Pathak and R. Sharma, “Smart Farming Using IoT and Cloud Computing,” *International Journal of Engineering Research & Technology*, vol. 9, no. 4, pp. 512–516, 2020.

- [17] S. Raza, A. A. Khan, and M. S. Khan, "Weather Forecasting Using Machine Learning Techniques," *International Journal of Computer Applications*, vol. 182, no. 12, pp. 25–30, 2018.
- [18] T. Wolf et al., "Transformers: State-of-the-Art Natural Language Processing," in *Proceedings of EMNLP*, 2020, pp. 38–45.
- [19] Open-Meteo, "Free Weather API for Developers," [Online]. Available: <https://open-meteo.com/>
- [20] Google, "Firebase Realtime Database Documentation," [Online]. Available: <https://firebase.google.com/docs>